PROJET YOON — VOYAGES EN CAR AU SÉNÉGAL

Déploiement sur AWS et sécurisation HTTPS par une PKI EJBCA / SoftHSM

Rapport technique, étape par étape

Terraform · GitHub Actions · Docker · ACM · EJBCA · SoftHSM

Auteur	Moustapha
Compte AWS	231644789895, région eu-west-3 (Paris)
Dépôts	GestionVoyageBakend, GestionVoyageFrontend, pki-aws-terraform-rendezvous
Version	11 novembre 2026

Résumé

Ce document retrace, dans l'ordre réel, la mise en production de l'application Yoon : refonte de l'application, authentification sans clé de GitHub vers AWS (OIDC), publication des images Docker dans ECR, déploiement des serveurs avec Terraform, puis sécurisation en HTTPS avec une PKI personnelle EJBCA dont les clés des autorités sont protégées par SoftHSM. Sans nom de domaine, le certificat est émis pour le nom DNS de l'Application Load Balancer, importé dans ACM, et ses listes de révocation sont publiées sur S3.

Chaque étape donne les fichiers, les commandes exécutées, les résultats obtenus, les captures d'écran et les valeurs choisies. Les erreurs rencontrées sont conservées avec leur correction : elles indiquent ce qu'il faut vérifier la prochaine fois.

Résultat final — `https://alb-moustapha-dev-2014044716.eu-west-3.elb.amazonaws.com` est servi en TLS 1.3 avec un certificat RSA 3072 émis par `Lab-IntermediateCA-SoftHSM2`. La Root est reconnue par Windows et Firefox. Le scanner de billets fonctionne avec la caméra d'un téléphone Android.

Étape	Contenu	État
0	OIDC GitHub → AWS	**OK**
1–3	ECR, images, serveurs Docker, RDS, ALB	**OK**
4a–4b	bucket des CRL, réglages des CA, profils EJBCA	**OK**
4c–4d	clé, CSR, émission, import ACM	**OK**
4e	listener 443, redirection HTTP → HTTPS	**OK**
4f	Root sur Windows et Android, test du scanner	**OK**
Suite	délai de paiement, mot de passe de la base, script de réémission	à faire

Table des matières

Résumé **i**

I Contexte et déploiement de l'application **1**

1 Le projet Yoon **2**
1.1 Objectif . . . 2
1.2 Les trois dépôts GitHub . . . 2
1.3 Comptes de démonstration . . . 3

2 Architecture et méthode de travail **4**
2.1 Architecture AWS . . . 4
2.2 Deux façons d'appliquer Terraform . . . 4
2.3 Méthode Git pour chaque changement . . . 5

3 Authentification GitHub vers AWS (OIDC) **6**
3.1 Le piège du `sub` . . . 6

4 Étape 1 : registres ECR et rôle de déploiement **7**

5 Étape 2 : images Docker publiées dans ECR **8**

6 Étape 3 : serveurs prêts pour Docker **9**
6.1 Incident du premier apply : erreur 502 . . . 9

II PKI EJBCA / SoftHSM et HTTPS **11**

7 Notions et décisions **12**

8 Code Terraform de l'étape 4 et bucket des CRL **13**
8.1 Pull request #15 . . . 13
8.2 Création du bucket (bootstrap, en local) . . . 13

9 Configuration d'EJBCA **15**
9.1 État initial . . . 15
9.2 Réglages de la Root CA . . . 15
9.3 Réglages de l'Intermediate CA . . . 17
9.4 Génération et récupération des CRL . . . 18

10 Partage de fichiers, publication S3, incident de la VM **19**
10.1 Dossier partagé VirtualBox . . . 19
10.2 Publication sur S3 . . . 19
10.3 Incident de la VM . . . 19

11 Profils EJBCA **22**
11.1 Profil de certificat `TLS-SERVER-YOON` . . . 22
11.2 Profil d'entité `EE-TLS-YOON` . . . 24

12 La CLI AWS dans la VM **27**

13 Le certificat de l'ALB **28**
13.1 Le nom de l'ALB a changé deux fois . . . 28
13.2 Nouvelle CRL par la ligne de commande d'EJBCA . . . 28
13.3 Clé RSA 3072 et CSR . . . 29
13.4 Émission par la ligne de commande d'EJBCA . . . 29
13.5 Trois émissions successives . . . 31
13.6 Vérification et import dans ACM . . . 32

14 Activation du port 443 **33**
14.1 Pull request #16 . . . 33
14.2 Tests depuis la VM . . . 33

15 Confiance dans la Root CA et test final **35**
15.1 Windows . . . 35
15.2 Vérification dans Firefox . . . 35
15.3 Téléphone Android et scanner . . . 37

III Sécurité, problèmes connus et exploitation **39**

16 Problèmes connus **40**
16.1 Délai de paiement dépassé dès le départ . . . 40
16.2 Mot de passe de la base dans `terraform.tfvars` . . . 40
16.3 Autres points . . . 41

17 Exploitation **42**
17.1 Publier la CRL chaque semaine . . . 42
17.2 Si dev est recréé . . . 42
17.3 Renouvellement . . . 42
17.4 Révoquer un certificat . . . 42

18 Règles de sécurité **43**

A Récapitulatif **44**

B Dépannage **45**

PARTIE I

Contexte et déploiement de l'application

1 Le projet Yoon

1.1 Objectif

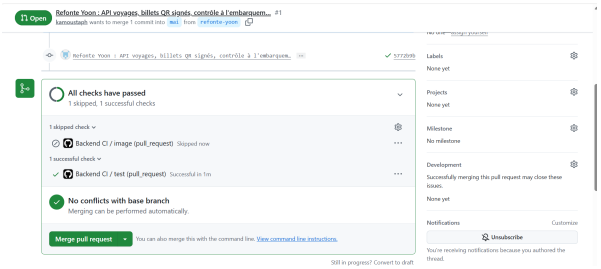
Yoon est une application de réservation de voyages en car au Sénégal. Le voyageur réserve un départ, paie par mobile money et reçoit un billet à QR code signé. À l'embarquement, l'agent scanne le billet avec la caméra de son téléphone. L'administrateur programme les départs et suit les contrôles.

Le travail décrit dans ce document couvre trois objectifs :

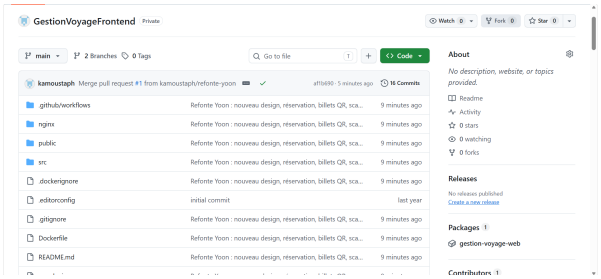
1. rendre l'application fonctionnelle et soignée (backend Spring Boot, frontend Angular) ;
2. la déployer sur AWS avec Terraform et GitHub Actions ;
3. la sécuriser en HTTPS avec **nos propres certificats**, émis par une PKI EJBCA dont les clés sont protégées dans SoftHSM, **sans acheter de nom de domaine**.

1.2 Les trois dépôts GitHub

Dépôt	Contenu	Rôle AWS
`GestionVoyageBakend`	API Spring Boot 3.5 (Java 17), JWT, billets QR signés HMAC-SHA256, profil `prod` pour MySQL	`yoon-app-deploy-role`
`GestionVoyageFrontend`	Angular 20, scanner QR (caméra + jsQR), image nginx qui relaie `/api`	`yoon-app-deploy-role`
`pki-aws-terraform-rendezvous`	Terraform : bootstrap, modules (vpc, security, ec2, rds, alb, app), environnements dev / staging / prod	`github-actions-role`

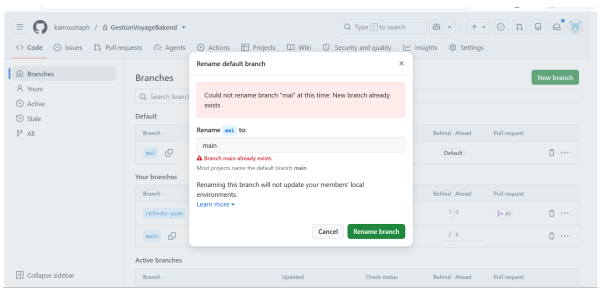


(a) Fusion de la refonte du backend (PR #1) : tests passés.

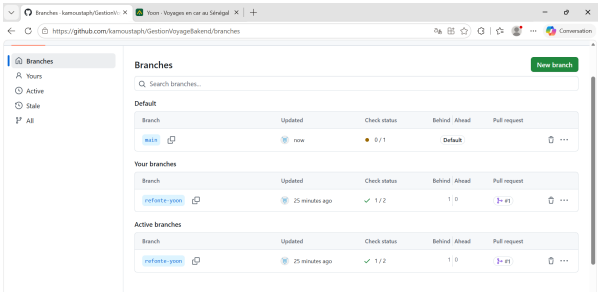


(b) Dépôt frontend après la refonte.

Branche par défaut — Le backend avait une branche `mai` au lieu de `main`. GitHub refuse de renommer une branche par défaut vers un nom déjà existant : il a fallu supprimer l'ancienne `main` (ses 8 commits restaient récupérables à `f967966`), puis renommer.



(a) Renommage refusé : « Branch main already exists ».



(b) Résultat : `main` est la branche par défaut.

1.3 Comptes de démonstration

En dev, le backend crée trois comptes au premier démarrage. Les départs de démonstration ne sont créés que si `DEMO_VOYAGES=true` (défaut `false` avec le profil `prod`).

Rôle	Email	Mot de passe
Voyageur	`voyageur@yoon.sn`	`Voyageur123!`
Agent de contrôle	`agent@yoon.sn`	`Agent123!`
Administrateur	`admin@yoon.sn`	`Admin123!`

Ne jamais — Garder ces mots de passe publics en usage réel : l'administrateur doit les changer avant la mise en service.

2 Architecture et méthode de travail

2.1 Architecture AWS

Ressource	Rôle
ALB `alb-moustapha-dev`	point d'entrée public, port 80 puis 443 (TLS)
2 EC2 (`ec2-1`, `ec2-2`)	conteneurs `web` (nginx, port 80) et `api` (Spring Boot, port 8093 interne)
RDS MySQL	base de données, sous-réseau privé
ECR	images `gestion-voyage-api` et `gestion-voyage-web`
SSM Parameter Store	secrets et paramètres `/yoon/<env>/*` ; commandes de déploiement à distance
ACM	certificat TLS de l'ALB, émis par EJBCA et importé
S3 `terraform-bucket-moustapha-2026`	états Terraform des environnements
S3 `pki-moustapha-2026`	publication publique (HTTP) des certificats de CA et des CRL

Séparer infrastructure et application — Terraform crée ce qui dure (réseau, serveurs, base, rôles). Les dépôts d'application publient des images ; changer une page Angular ne déclenche jamais de `terraform apply`, et un dépôt d'application ne peut pas modifier l'infrastructure.

2.2 Deux façons d'appliquer Terraform

Dossier	État	Application	Contenu
`bootstrap/`	local	depuis le PC	bucket d'état, OIDC, rôles, ECR, bucket PKI
`environments/<env>/`	S3	GitHub Actions	VPC, EC2, RDS, ALB, SSM

La pipeline `.github/workflows/terraform.yml` lance un `plan` des trois environnements sur chaque pull request, et un `apply` ou `destroy` d'un environnement à la demande (*Run workflow*).

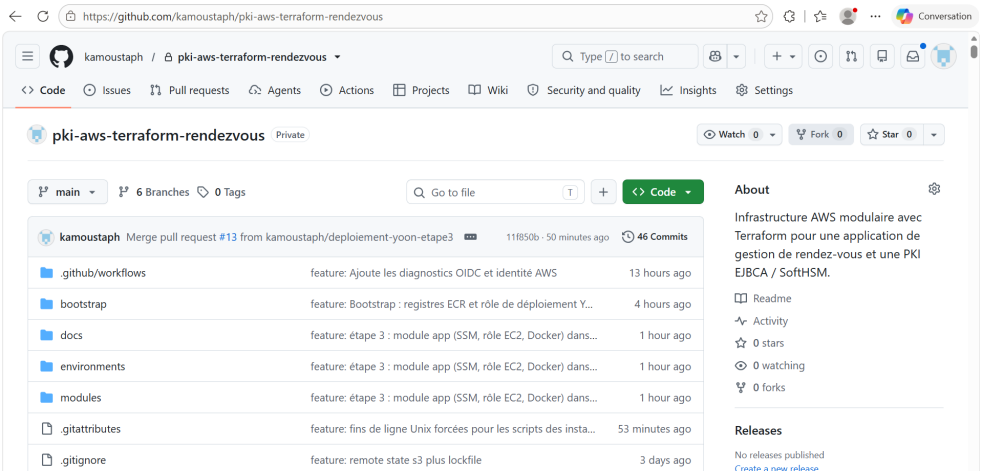


Fig. 2.1 : Le dépôt Terraform : bootstrap, modules, environnements, docs.

2.3 Méthode Git pour chaque changement

1. Partir de `main` à jour : `git fetch origin` puis `git switch -c <branche> origin/main`.
2. Intégrer un zip **fichier par fichier**, avec sauvegarde des fichiers remplacés (jamais de dossier écrasé).
3. Vérifier : `git status --short`, `git diff --stat`, environnements identiques, `terraform fmt`, `terraform validate`, absence de secret.
4. Ajouter des fichiers précis (jamais `git add .`), commiter.
5. `git push -u origin <branche>` : le `-u` fait suivre la bonne branche distante.
6. Lire les plans de la pull request, fusionner, puis *Run workflow* → `apply`.

```
git diff --no-index environments/dev/main.tf environments/prod/main.tf   # rien =
↪  identiques
terraform -chdir=environments/dev init -backend=false
terraform -chdir=environments/dev validate
git diff | Select-String -Pattern 'BEGIN .*PRIVATE
↪  KEY|password\s*=\s*"|secret\s*=\s*"'
```

Pièges rencontrés —

- Une branche déjà fusionnée ne propose plus de pull request : repartir de `origin/main`.
- Dans PowerShell, un motif contenant `"` se met entre guillemets simples.
- `init` dans un module crée `.terraform/` : ne pas le commiter ; fermer VS Code s'il est verrouillé.

3 Authentification GitHub vers AWS (OIDC)

Aucune clé AWS n'est stockée dans GitHub. Chaque workflow demande un jeton OIDC de courte durée, qu'AWS échange contre des identifiants temporaires si le jeton correspond à la *trust policy* du rôle.

3.1 Le piège du `sub`

Ce compte envoie le claim `sub` avec les **identifiants numériques** du propriétaire et du dépôt :

```
repo:kamoustaph@162557925/pki-aws-terraform-rendezvous@1433162106:pull_request
```

Une trust policy écrite au format classique `repo:kamoustaph/pki-aws-terraform-rendezvous:pull_request` est refusée avec `Not authorized to perform sts:AssumeRoleWithWebIdentity`. La correction a consisté à afficher le `sub` réel dans la pipeline (étape « Debug OIDC (sub) »), puis à l'écrire tel quel dans la trust policy.

Rôle	Contextes acceptés
`github-actions-role`	dépôt Terraform : `pull_request`, `environment:dev|staging|prod`, `ref:refs/heads/main`
`yoon-app-deploy-role`	dépôts Bakend et Frontend : `ref:refs/heads/main` uniquement

Ne jamais — Renommer ou recréer un dépôt : son identifiant change et l'accès AWS est cassé.

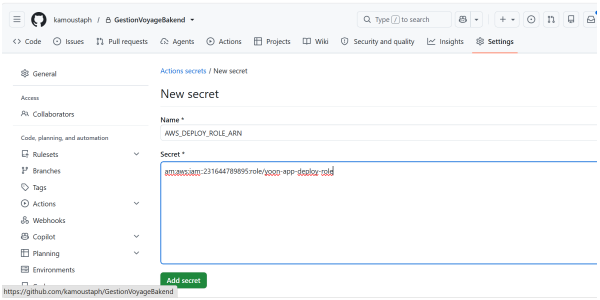
4 Étape 1 : registres ECR et rôle de déploiement

Appliquée en local depuis `bootstrap/` (résultat : `6 added, 1 changed, 0 destroyed`).

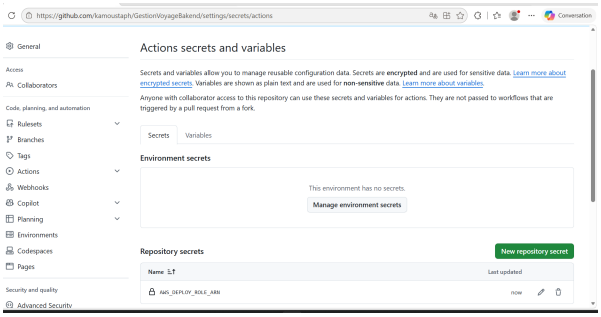
Fichier	Contenu
`bootstrap/ecr.tf`	registres `gestion-voyage-api` et `gestion-voyage-web`, scan au push, AES256, 15 images conservées
`bootstrap/deploy_role.tf`	`yoon-app-deploy-role` : push/pull ECR, `ssm:SendCommand` limité aux instances `Project=moustapha`
`bootstrap/oidc.tf`	ajout de `ssm:*` et `ecr:*` à la pipeline Terraform
`bootstrap/outputs.tf`	sorties `ecr_repository_urls`, `app_deploy_role_arn`

```
cd C:\terraform_modules_moustapha\bootstrap
terraform plan      # 6 to add, 1 to change, 0 to destroy
terraform apply
```

Le secret `AWS_DEPLOY_ROLE_ARN` est ensuite créé dans les deux dépôts d'application. Ce n'est qu'un ARN de rôle, pas un identifiant secret.



(a) Création du secret `AWS_DEPLOY_ROLE_ARN`.

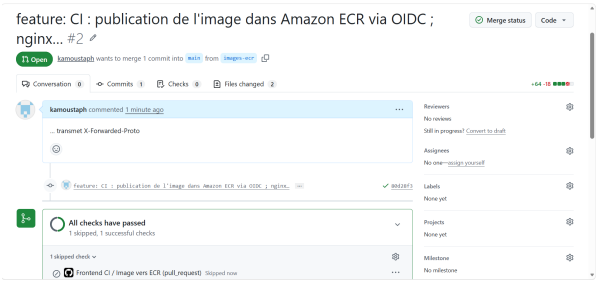


(b) Le secret enregistré dans le dépôt.

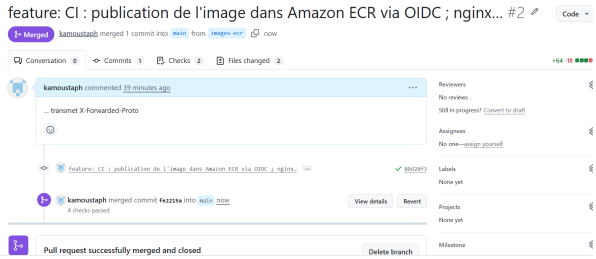
L'état du bootstrap est local — Après chaque `apply` du bootstrap, copier `bootstrap\terraform.tfstate` dans une sauvegarde, et hors du PC.

5 Étape 2 : images Docker publiées dans ECR

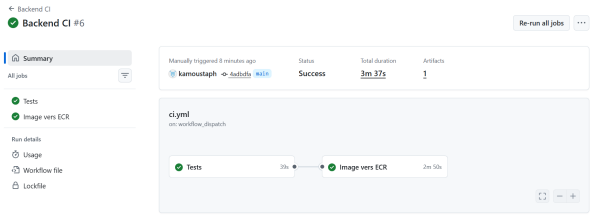
À chaque fusion sur `main`, le workflow `ci.yml` de chaque dépôt d'application teste, construit l'image et la pousse dans ECR avec deux tags : le SHA du commit et `latest`. Sur une pull request, le job *Image vers ECR* est volontairement ignoré (le rôle n'accepte que `main`).



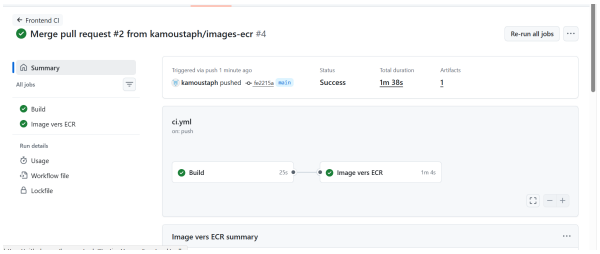
(a) PR #2 du frontend : publication vers ECR via OIDC, et transmission de `X-Forwarded-Proto`.



(b) PR #2 fusionnée.



(a) Backend : tests (3 min 37) puis image vers ECR.



(b) Frontend : build puis image vers ECR.

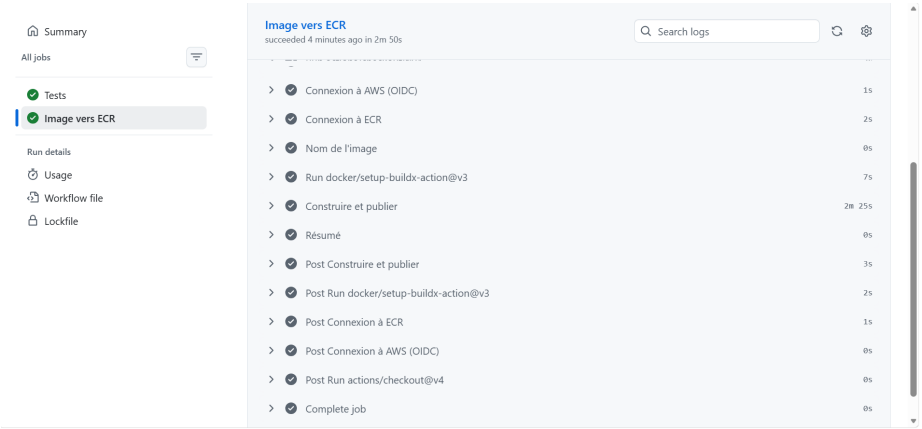


Fig. 5.3 : Détail du job « Image vers ECR » : connexion AWS par OIDC, connexion ECR, construction et publication.

6 Étape 3 : serveurs prêts pour Docker

Au premier démarrage, chaque EC2 installe Docker, lit sa configuration dans SSM, tire les images depuis ECR et lance `api`, attend qu'elle soit prête, puis lance `web`.

Fichier	Contenu
`modules/app/main.tf`	secrets JWT et QR générés, paramètres SSM, rôle et profil IAM des EC2, user data `primary`/`secondary`
`modules/app/user_data.sh.tftpl`	swap, Docker, AWS CLI, agent SSM ; attente de `/yoon/<env>/db_url`
`modules/app/deploy.sh`	lecture SSM, `docker pull`, lancement `api` puis `web`
`modules/ec2/*`	profil IAM, IMDSv2 obligatoire, disque gp3 chiffré
`environments/*/main.tf`	module `app`, `depends_on = [module.app]` sur les EC2

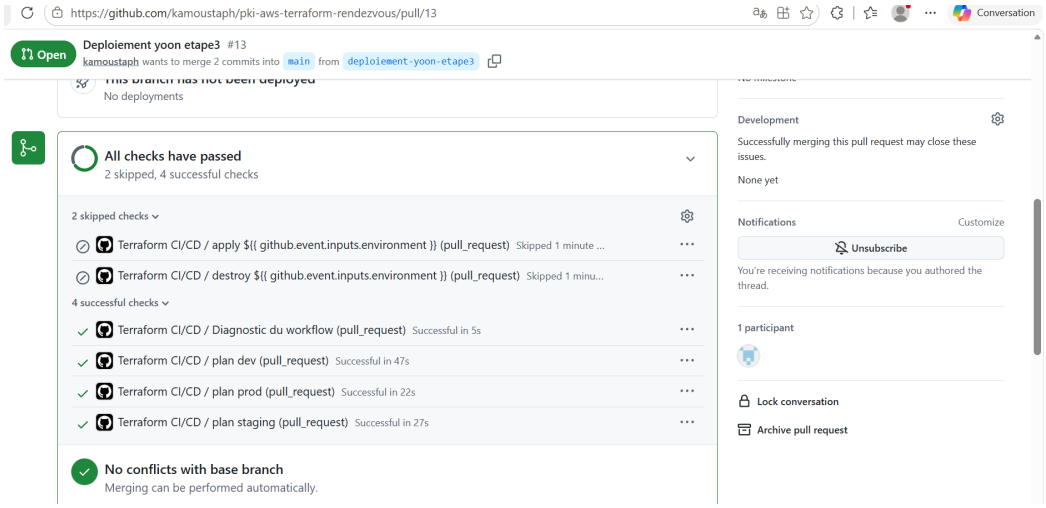
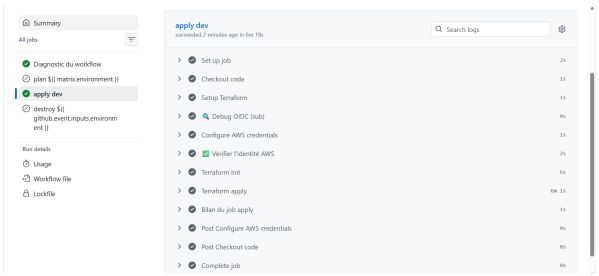


Fig. 6.1 : PR #13 : plans dev, staging et prod réussis.

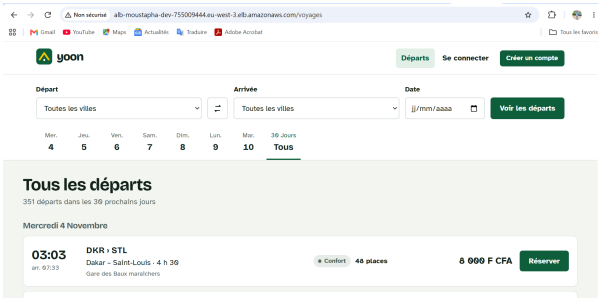
6.1 Incident du premier apply : erreur 502

Les EC2 ont été créées avant la fin de RDS, donc avant les paramètres SSM : `deploy.sh` n'a rien lancé. Rattrapage par SSM Run Command, puis correctif permanent (PR #14) : `depends_on` et boucle d'attente.

```
aws ssm send-command --region eu-west-3 --document-name AWS-RunShellScript `
  --instance-ids <id-ec2-1> --parameters 'commands=/opt/yoon/deploy.sh all latest'
```



(a) *apply dev* de l'étape 3.



(b) Yoon en ligne derrière l'ALB, encore en HTTP : 351 départs de démonstration.

PARTIE II

PKI EJBCA / SoftHSM et HTTPS

7 Notions et décisions

HTTPS est obligatoire pour Yoon : les navigateurs refusent l'accès à la caméra (scanner de billets) sur une page HTTP. Le certificat est émis par notre PKI, sans acheter de domaine.

Terme	Sens dans ce projet
Root CA	`Lab-RootCA-SoftHSM2` ; clé dans SoftHSM ; seul certificat installé sur les appareils
Intermediate CA	`Lab-IntermediateCA-SoftHSM2` ; signée par la Root ; signe le certificat de l'ALB
CSR	demande de certificat : clé publique et nom, signée par la clé privée qui reste chez nous
SAN	nom DNS comparé par le navigateur à l'adresse visitée
CRL	liste signée des certificats révoqués, avec une date *Next Update*
CDP / AIA	adresses, inscrites dans chaque certificat, de la CRL et du certificat de l'autorité émettrice
ACM	service AWS qui conserve le certificat et sa clé et les fournit à l'ALB

Le certificat porte le nom DNS de l'ALB — ACM accepte d'importer un certificat d'une CA privée pour n'importe quel nom. Mais ce nom change à chaque recréation de l'ALB (`destroy` puis `apply`) : il faut alors réémettre. C'est arrivé deux fois pendant ce travail.

Pas de certificat générique *.eu-west-3.elb.amazonaws.com — Il couvrirait tous les ALB de la région, y compris ceux d'autres clients : sur un appareil qui fait confiance à notre Root, il permettrait de les usurper. Un certificat ne couvre que des noms que l'on contrôle.

CRL en HTTP, dans un bucket du bootstrap — Les adresses sont figées dans les certificats : elles doivent être permanentes, donc indépendantes de l'ALB. Vérifier la révocation d'un certificat HTTPS ne doit pas dépendre d'un autre certificat HTTPS ; la CRL est signée.

Clé de l'ALB créée par OpenSSL, importée par la CLI — ACM exige la clé privée en clair, ce qu'un HSM refuse de fournir : SoftHSM protège les clés *des CA*. L'import se fait par `aws acm import-certificate` et non par Terraform, sinon la clé serait dans l'état Terraform.

8 Code Terraform de l'étape 4 et bucket des CRL

8.1 Pull request #15

Fichier	Changement
`bootstrap/pki_publication.tf`	bucket `pki-moustapha-2026` : versionnage, AES256, ACL bloquées, lecture publique de `crl/*` et `ca/*` uniquement
`bootstrap/outputs_pki.tf`	sortie `pki_publication` (4 adresses)
`modules/alb/*`	`certificate_arn` (défaut `null`) : listener 443 et redirection 80 → 443
`environments/*/`	variable `certificate_arn`, sortie `load_balancer_dns`

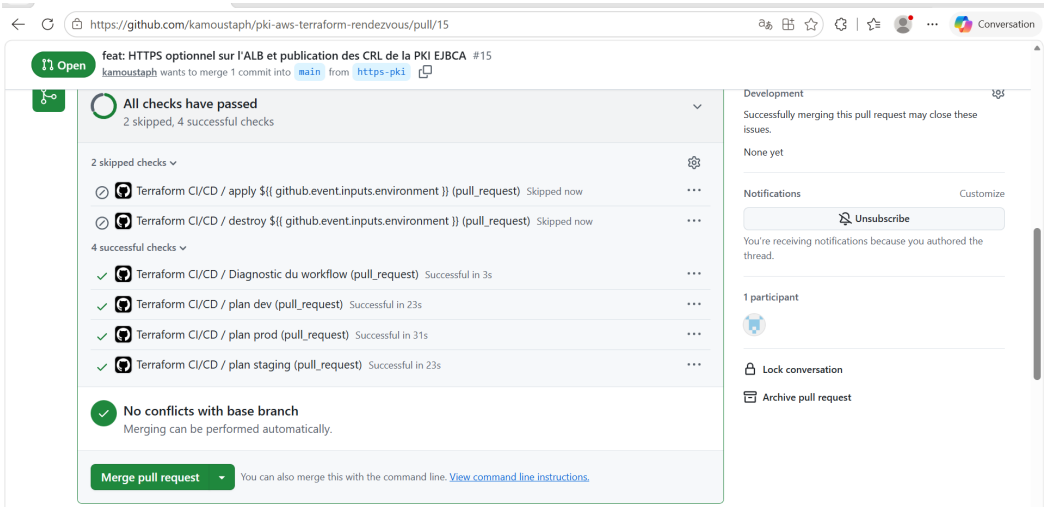


Fig. 8.1 : PR #15 : plans sans changement, car `certificate_arn` vaut `null`. Le module est rétrocompatible.

8.2 Création du bucket (bootstrap, en local)

```
cd C:\terraform_modules_moustapha\bootstrap
terraform plan      # Plan: 5 to add, 0 to change, 0 to destroy
terraform apply     # Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
terraform output pki_publication
```

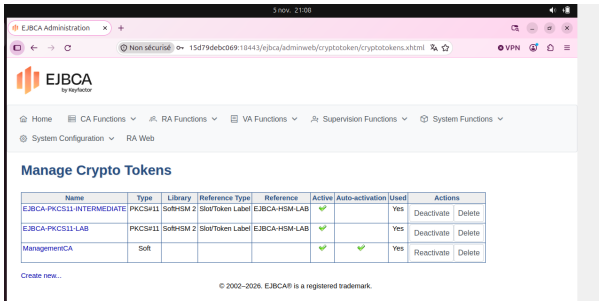
Sortie	Adresse
`root_crl_cdp`	`http://pki-moustapha-2026.s3.eu-west-3.amazonaws.com/crl/root-ca.crl`
`root_cert_aia`	`http://pki-moustapha-2026.s3.eu-west-3.amazonaws.com/ca/root-ca.crt`
`intermediate_crl_cdp`	`http://pki-moustapha-2026.s3.eu-west-3.amazonaws.com/crl/intermediate-ca.crl`
`intermediate_cert_aia`	`http://pki-moustapha-2026.s3.eu-west-3.amazonaws.com/ca/intermediate-ca.crt`

```
curl.exe -sI http://pki-moustapha-2026.s3.eu-west-3.amazonaws.com/      -> 403
↪  (listage interdit)
```

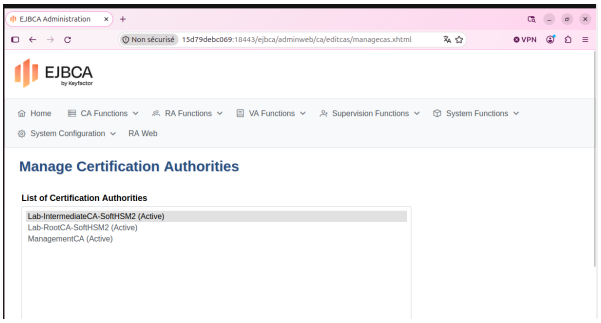
9 Configuration d'EJBCA

9.1 État initial

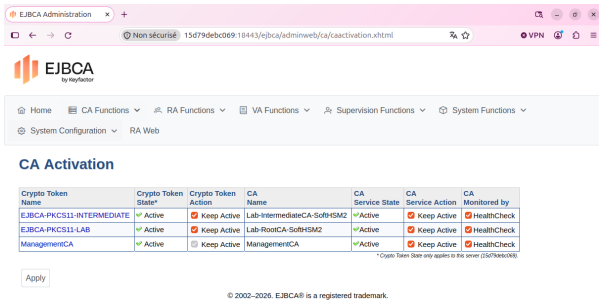
EJBCA tourne dans Docker sur une VM Ubuntu (VirtualBox), conteneurs `ejbca-hsm-lab-ejbca-1` et `ejbca-hsm-lab-mariadb-1`. Interface : `https://15d79debc069:18443/ejbca/adminweb/`.



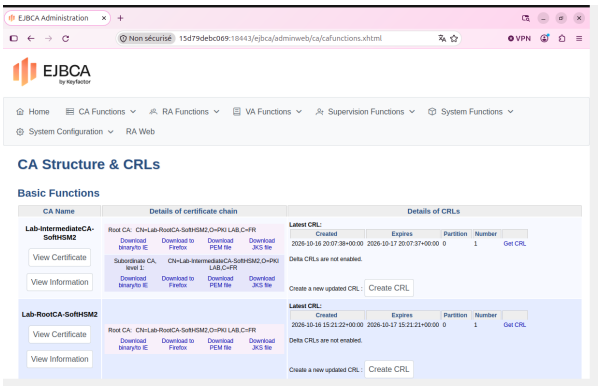
(a) Jetons PKCS#11 SoftHSM : `EJBCA-PKCS11-LAB` (Root) et `EJBCA-PKCS11-INTERMEDIATE`.



(b) Les trois CA actives.



(a) Activation : jetons et CA actifs, surveillés.



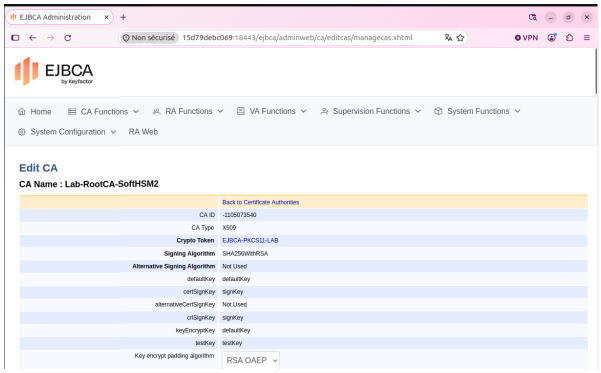
(b) CRL n° 1, expirées depuis le 17/10/2026 : le service de mise à jour ne tournait pas.

CA	Certificat	Jeton
Root	`CN=Lab-RootCA-SoftHSM2,O=PKI LAB,C=FR`, autosignée, profil ROOTCA, jusqu'au 13/10/2036	`EJBCA-PKCS11-LAB`
Intermediate	`CN=Lab-IntermediateCA-SoftHSM2,O=PKI LAB,C=FR`, profil SUBCA, jusqu'au 15/10/2031	`EJBCA-PKCS11-INTERMEDIATE`

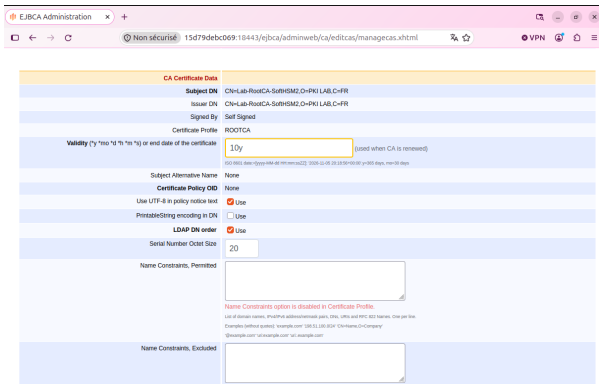
Les clés ne sont pas exportables (« This CA's Crypto Token does not support export ») : c'est voulu.

9.2 Réglages de la Root CA

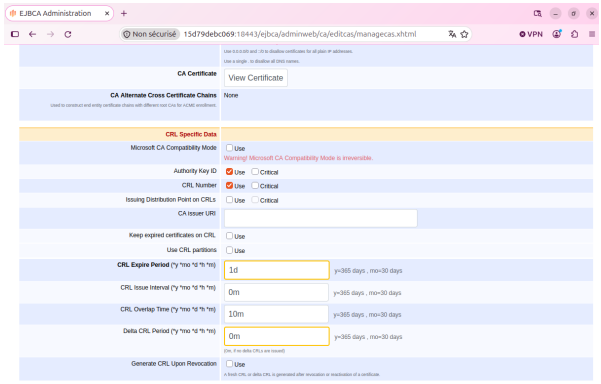
Menu *CA Functions* → *Certification Authorities* → *Lab-RootCA-SoftHSM2* → *Edit CA*.



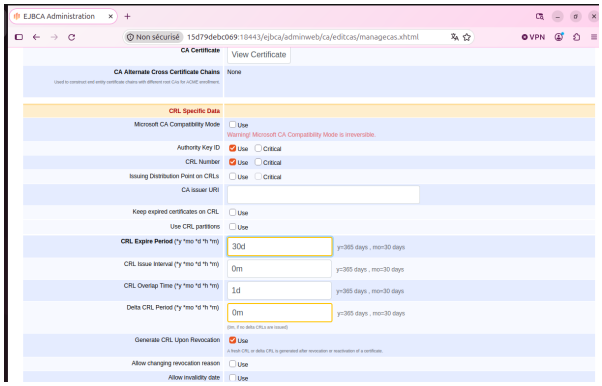
(a) Jeton et clés de signature.



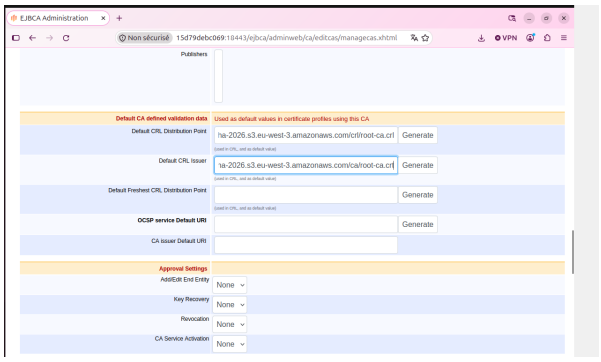
(b) Données du certificat de la Root.



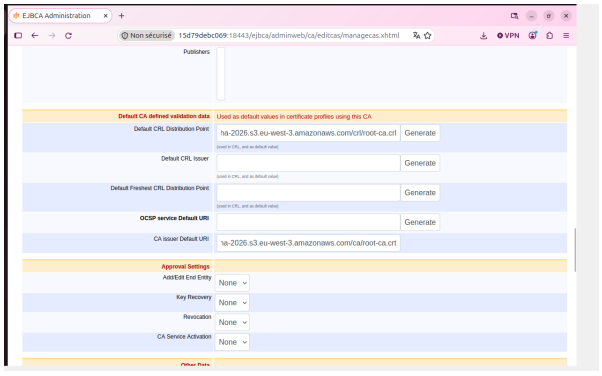
(a) Avant : CRL valable 1 jour.



(b) Après : 30 jours, chevauchement 1 jour, CRL générée à chaque révocation.



(a) Erreur : l'URL du certificat était saisie dans *Default CRL Issuer*.

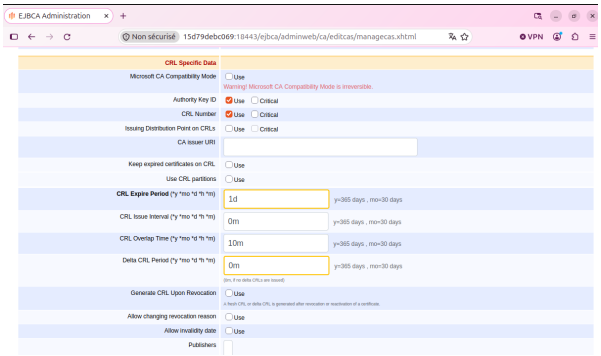


(b) Corrigé : CRL Issuer vide, URL dans *CA issuer Default URI*.

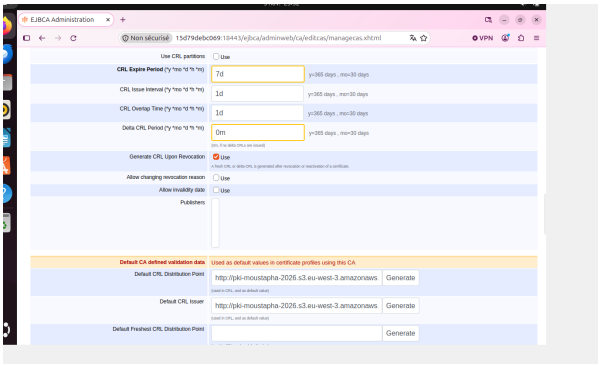
Champ (Root)	**Valeur finale**
CRL Expire Period / Issue Interval / Overlap	`30d` / `0m` / `1d`
Generate CRL Upon Revocation	coché
Default CRL Distribution Point	`.../crl/root-ca.crl`
Default CRL Issuer	**vide**
CA issuer Default URI	`.../ca/root-ca.crt`

Trois champs différents — *Default CRL Distribution Point* donne l'adresse de la CRL ; *Default CRL Issuer* désigne une autre autorité qui signerait les CRL (CRL indirecte) et doit rester vide ; *CA issuer Default URI* donne l'adresse du certificat de la CA.

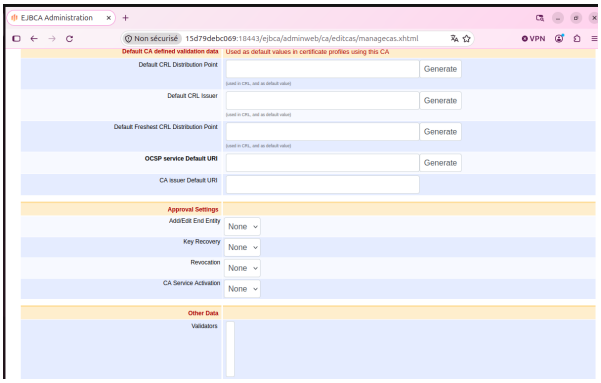
9.3 Réglages de l'Intermediate CA



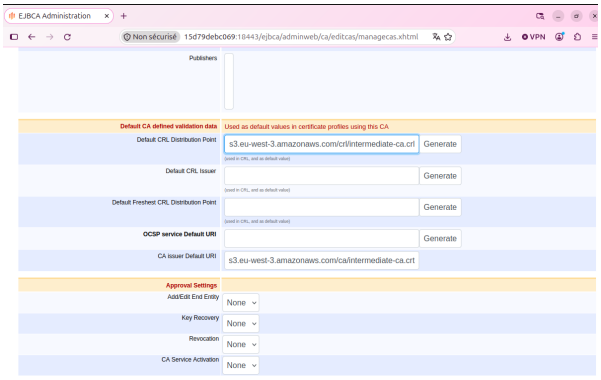
(a) Avant : 1 jour.



(b) Après : 7 jours, émission chaque jour.



(a) Avant : adresses vides.

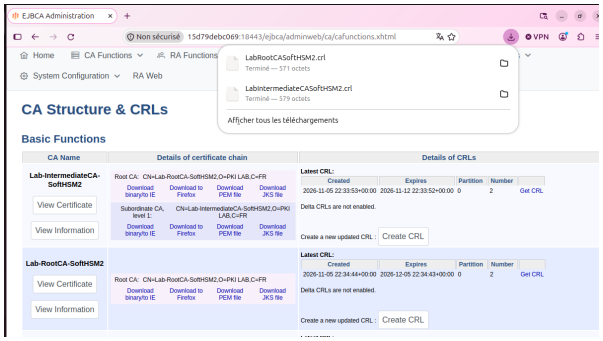


(b) Final : CDP et AIA de l'Intermediate.

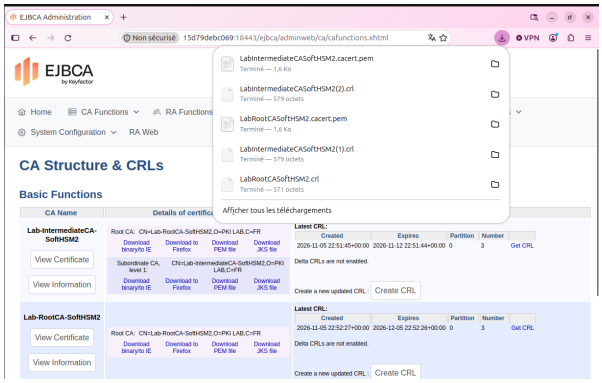
Champ (Intermediate)	Valeur finale
CRL Expire Period / Issue Interval / Overlap	`7d` / `1d` / `1d`
Default CRL Distribution Point	`.../crl/intermediate-ca.crl`
Default CRL Issuer	**vide**
CA issuer Default URI	`.../ca/intermediate-ca.crt`

L'Intermediate lui-même n'a ni CDP ni AIA — Il a été émis avant ces réglages (`No extensions in certificate`). La chaîne reste valide ; on le conserve.

9.4 Génération et récupération des CRL



(a) CRL n° 2 (avant correction du CRL Issuer).



(b) CRL n° 3, publiées.

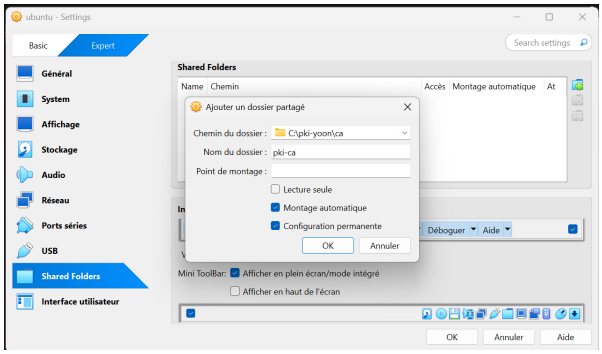
```
mkdir -p ~/pki-yoon/ca && cd ~/pki-yoon/ca
mv ~/Téléchargements/LabRootCASoftHSM2.cacert.pem          root-ca.pem
mv ~/Téléchargements/LabIntermediateCASoftHSM2.cacert.pem  intermediate-ca.pem
mv ~/Téléchargements/LabRootCASoftHSM2.crl          root-ca.crl
mv ~/Téléchargements/LabIntermediateCASoftHSM2.crl  intermediate-ca.crl
openssl x509 -in root-ca.pem         -outform DER -out root-ca.der
openssl x509 -in intermediate-ca.pem -outform DER -out intermediate-ca.der
openssl verify -CAfile root-ca.pem intermediate-ca.pem
```

```
intermediate-ca.pem: OK
root-ca.crl          crlNumber=0x03   nextUpdate=Dec  5 22:52:26 2026 GMT
intermediate-ca.crl  crlNumber=0x03   nextUpdate=Nov 12 22:51:44 2026 GMT
```

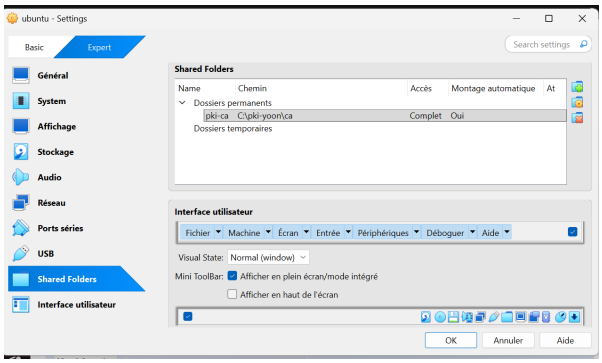
Fichier sensible rangé — `SuperAdmin.p12` (certificat et clé de l'administrateur EJBCA) traînait dans Téléchargements : déplacé dans `~/pki-lab-backups/admin/` avec les droits 700 et 600.

10 Partage de fichiers, publication S3, incident de la VM

10.1 Dossier partagé VirtualBox



(a) Ajout du dossier `pki-ca` → `C:\pki-yoon\ca`.



(b) Montage automatique et permanent.

```
sudo apt install -y build-essential dkms linux-headers-$(uname -r) bzip2
sudo sh /media/$USER/VBox_GAs_*/VBoxLinuxAdditions.run      # Additions invité (CD)
sudo usermod -aG vboxsf $USER && sudo reboot
cp ~/pki-yoon/ca/*.der ~/pki-yoon/ca/*.pem ~/pki-yoon/ca/*.crl /media/sf_pki-ca/
```

Seuls des fichiers **publics** passent par ce dossier : jamais de clé privée.

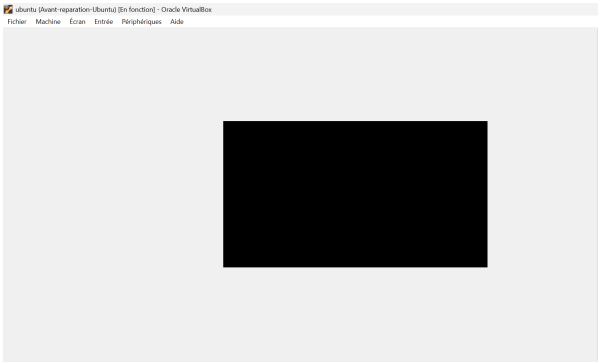
10.2 Publication sur S3

```
aws s3 cp root-ca.der         s3://pki-moustapha-2026/ca/root-ca.crt
↪  --content-type application/pkix-cert
aws s3 cp intermediate-ca.der s3://pki-moustapha-2026/ca/intermediate-ca.crt
↪  --content-type application/pkix-cert
aws s3 cp root-ca.crl         s3://pki-moustapha-2026/crl/root-ca.crl
↪  --content-type application/pkix-crl
aws s3 cp intermediate-ca.crl s3://pki-moustapha-2026/crl/intermediate-ca.crl
↪  --content-type application/pkix-crl
```

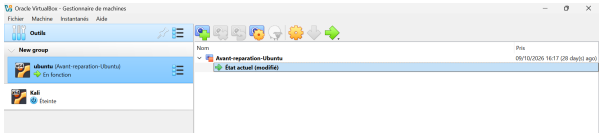
```
HTTP/1.1 200 OK   Content-Type: application/pkix-cert   x-amz-server-side-encryption:
↪  AES256
FC : aucune différence trouvée     (fichier servi par S3 identique à celui d'EJBCA)
```

10.3 Incident de la VM

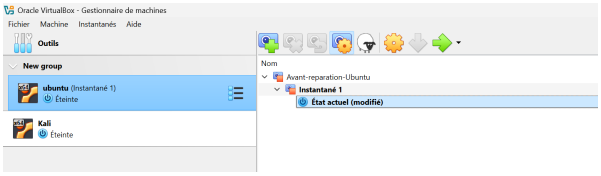
Les Additions invité installées depuis le CD ont rendu la VM instable : écran noir, disque incohérent, puis plantage du noyau.



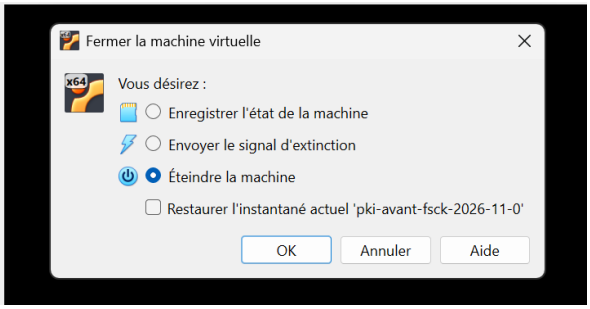
(a) Écran noir après GRUB.



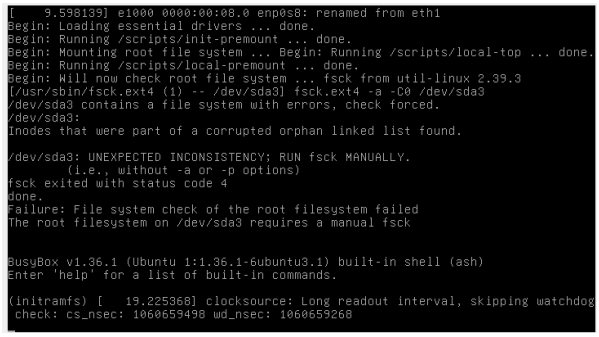
(b) Le seul instantané datait du 09/10, avant la création des CA : à ne pas restaurer.



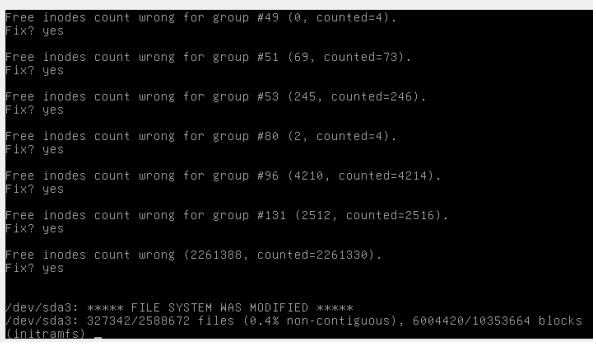
(a) Nouvel instantané de l'état actuel avant toute réparation.



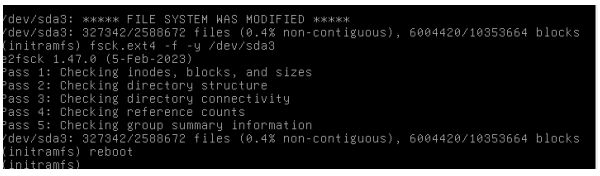
(b) Extinction sans restauration.



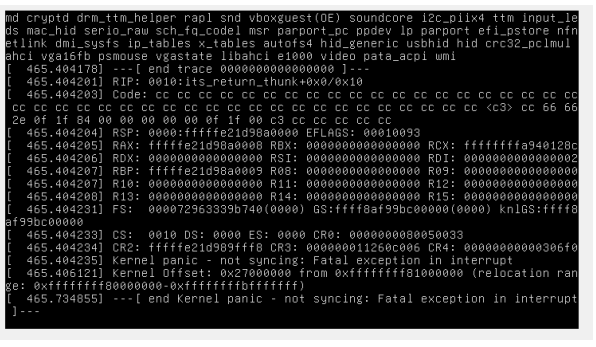
(a) `UNEXPECTED INCONSISTENCY; RUN fsck MANUALLY.`



(b) `fsck.ext4 -f -y /dev/sda3` : compteurs corrigés.



(a) Deuxième passage propre.



(b) Plantage du noyau, avec le module `vboxguest(OE)` chargé.



(a) Ligne `linux` dans l'éditeur GRUB (touche `e`).



(b) Ajout de `nomodeset module_blacklist=vboxguest`.

GRUB est en QWERTY — Sur un clavier AZERTY : `no,odeset` pour `nomodeset`, `,odule` pour `module`, Maj+) pour `_`, `blqcklist` pour `blacklist`.

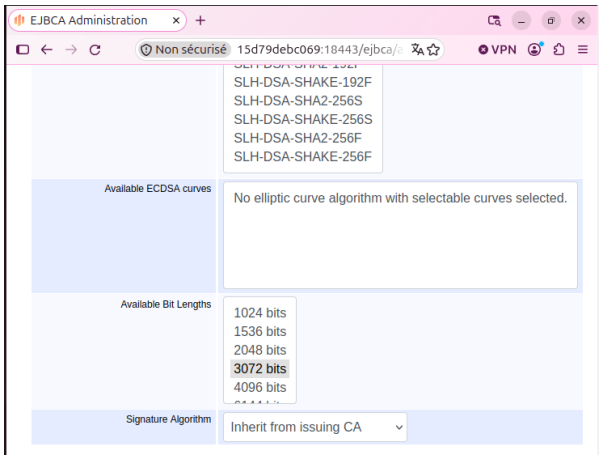
```
sudo /opt/VBoxGuestAdditions-*/uninstall.sh      # retirer les Additions du CD
```

Leçons — Instantané avant toute installation touchant le noyau ; vérifier la date d'un instantané avant de le restaurer ; éteindre proprement ; sauvegarder SoftHSM et la base EJBCA hors de la VM.

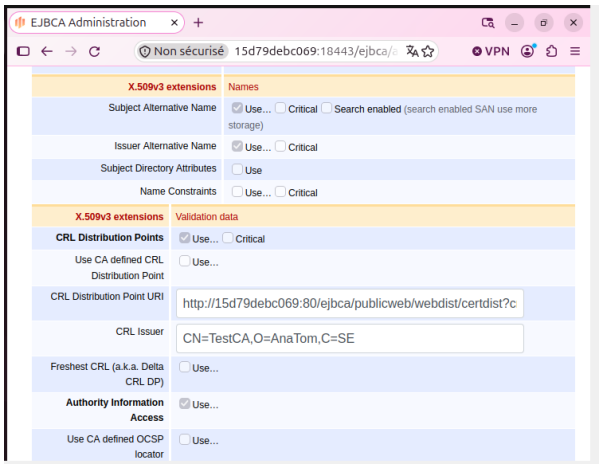
11 Profils EJBCA

11.1 Profil de certificat `TLS-SERVER-YOON`

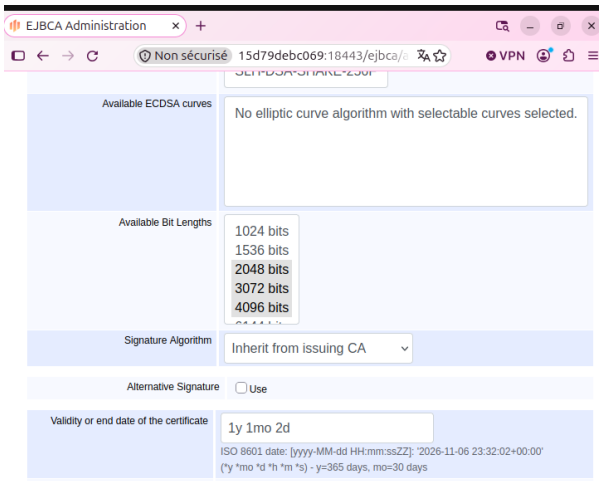
Clone du profil `SERVER`. Il a fallu plusieurs passes, et l'export XML a révélé des réglages affichés mais jamais enregistrés.



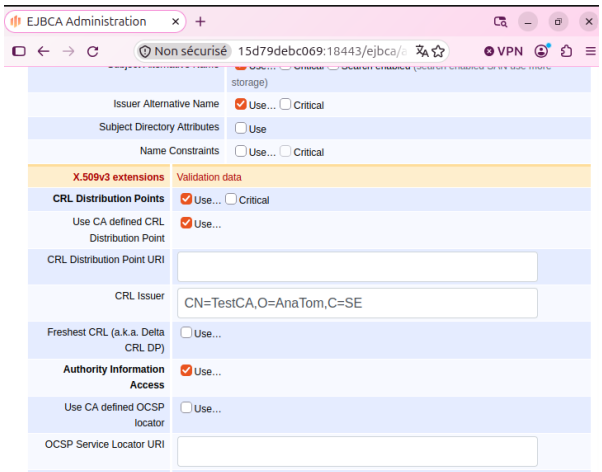
(a) Passe 1 : seulement 3072 bits.



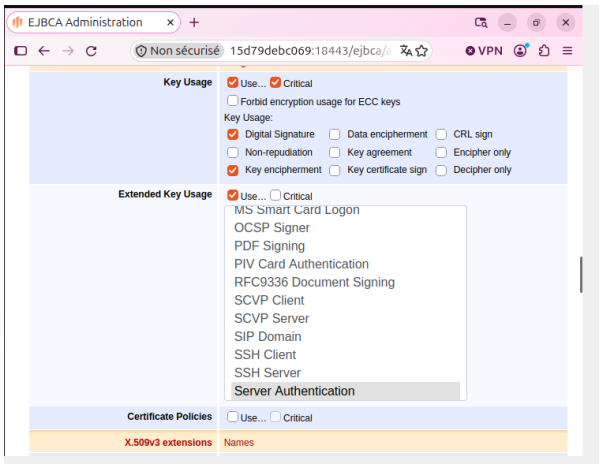
(b) Passe 1 : URI interne au conteneur, CRL Issuer d'exemple, *Use CA defined* décoché.



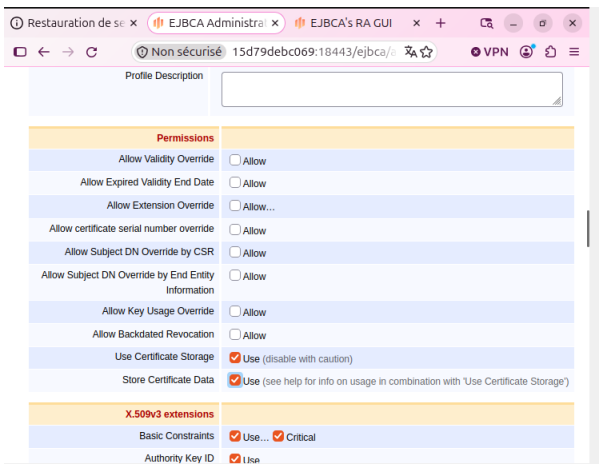
(a) 2048, 3072 et 4096 bits (Ctrl+clic).



(b) *Use CA defined CRL Distribution Point* coché.



(a) Key Usage critique et EKU *Server Authentication*.



(b) *Use Certificate Storage* et *Store Certificate Data* cochés.

11.1.1 Vérifier ce qui est réellement enregistré

```
docker exec ejbca-hsm-lab-ejbca-1 /opt/keyfactor/bin/ejbca.sh ca exportprofiles -d
↪  /tmp/profils
P=/tmp/profils/certprofile_TLS-SERVER-YOON-334309444.xml
docker exec ejbca-hsm-lab-ejbca-1 grep -i -A1 -E "<string>(usedefaultcrldistributionp
↪  oint|usecertificatestorage|storecertificatedata)</string>"
↪  $P
docker exec ejbca-hsm-lab-ejbca-1 grep -i -A12 "<string>availablebitlengths</string>"
↪  $P | grep "<int>"
```

```
moustapha@moustapha-VirtualBox:~/pki-yoon/ca$ docker exec -u root ejbca-hsm-lab-ejbca-1 rm -rf /tmp/profils
moustapha@moustapha-VirtualBox:~/pki-yoon/ca$ docker exec ejbca-hsm-lab-ejbca-1 mkdir -p /tmp/profils
moustapha@moustapha-VirtualBox:~/pki-yoon/ca$ docker exec ejbca-hsm-lab-ejbca-1 /opt/keyfactor/bin/ejbca.sh ca exportprofiles -d /tmp
/profils >/dev/null
moustapha@moustapha-VirtualBox:~/pki-yoon/ca$ P=/tmp/profils/certprofile_TLS-SERVER-YOON-334309444.xml
moustapha@moustapha-VirtualBox:~/pki-yoon/ca$ docker exec ejbca-hsm-lab-ejbca-1 grep -i -A1 -E "<string>(usecertificatestorage|storec
ertificatedata)</string>" $P
   <string>usecertificatestorage</string>
   <boolean>false</boolean>
--
   <string>storecertificatedata</string>
   <boolean>false</boolean>
moustapha@moustapha-VirtualBox:~/pki-yoon/ca$
```

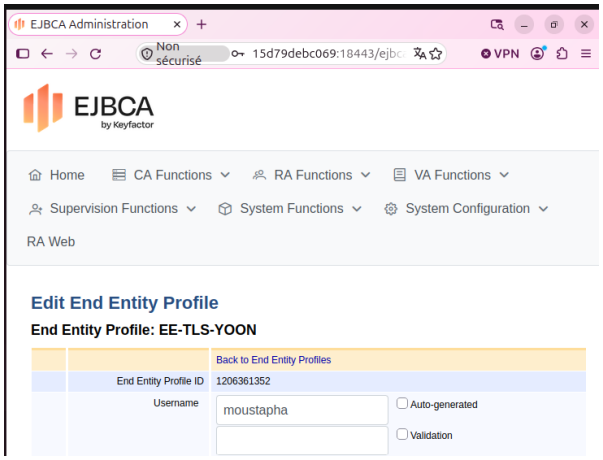
Fig. 11.4 : Diagnostic : stockage des certificats désactivé (`false`).

Champ	Valeur finale
Available Key Algorithms / Bit Lengths	RSA / 2048, 3072, 4096
Validity	`1y 1mo 2d` = 397 jours
Permissions *Allow … Override*	décochées
Use Certificate Storage / Store Certificate Data	**cochés**
Key Usage	critique : Digital Signature, Key Encipherment
Extended Key Usage	Server Authentication
CRL Distribution Points	coché + **Use CA defined CRL Distribution Point**
Authority Information Access	coché + Use CA defined CA issuer
Available CAs	`Lab-IntermediateCA-SoftHSM2` uniquement

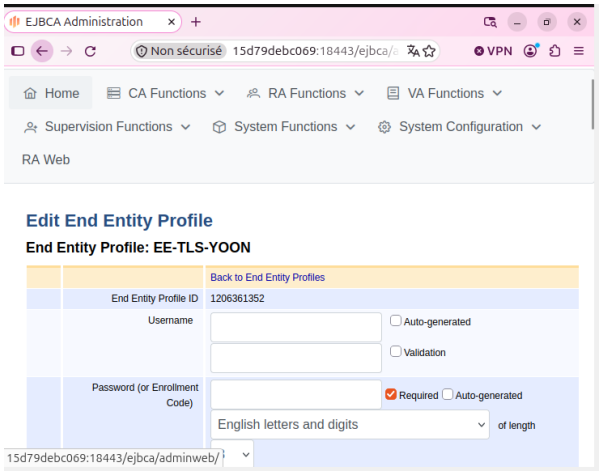
Trois erreurs révélées par l'export — 2048 bits non autorisé (`Illegal key length`) ; CDP absent du certificat (`usedefaultcrldistributionpoint=false`) ; certificats non

enregistrés, donc impossibles à révoquer (`usecertificatestorage=false`). Toujours cliquer sur *Save* tout en bas, puis vérifier par l'export.

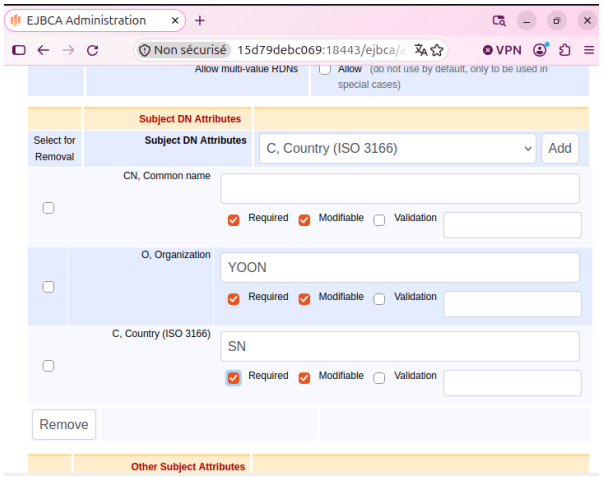
11.2 Profil d'entité `EE-TLS-YOON`



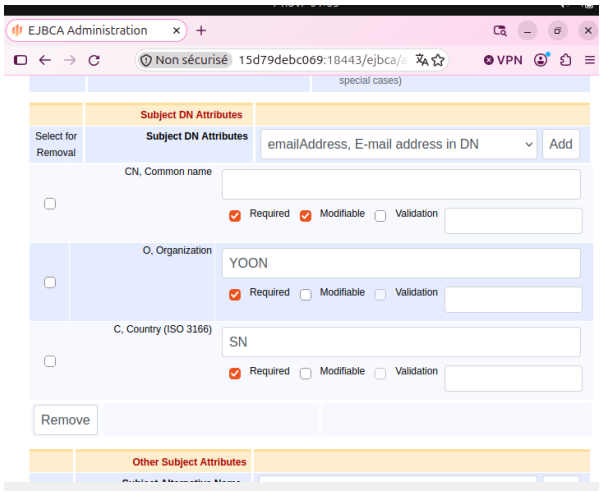
(a) Avant : username prérempli.



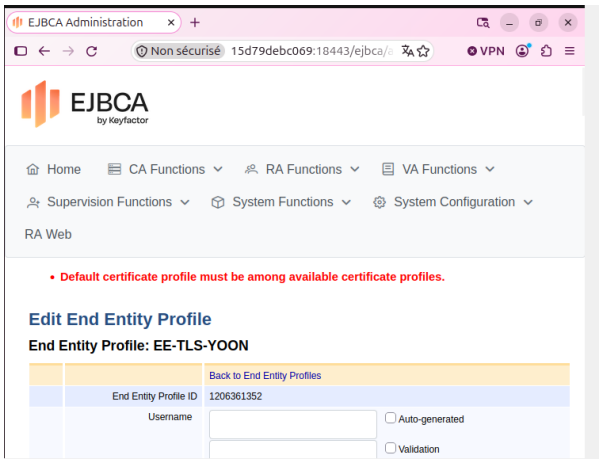
(b) Username et mot de passe vides.



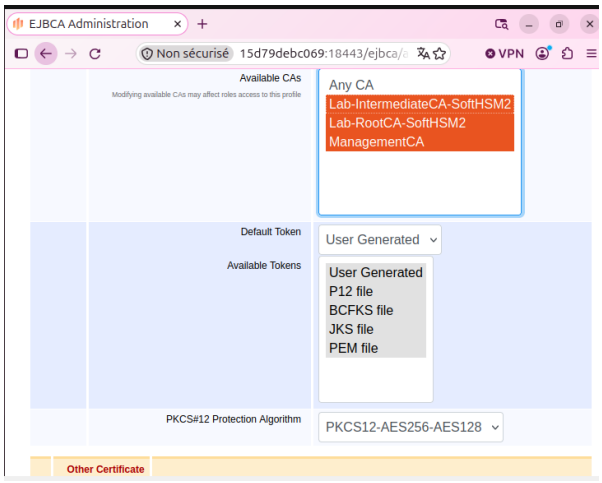
(a) O et C modifiables : à corriger.



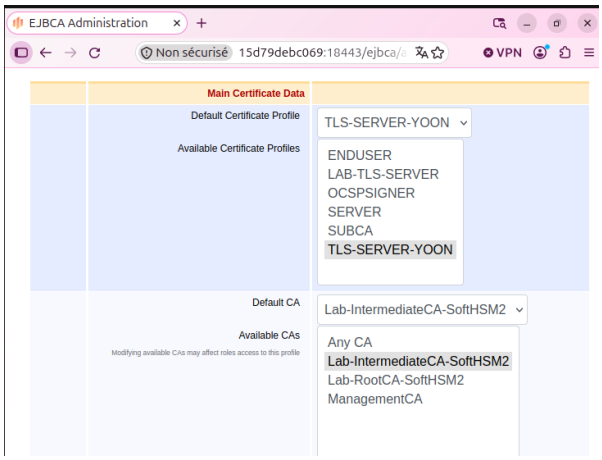
(b) O = `YOON`, C = `SN`, non modifiables.



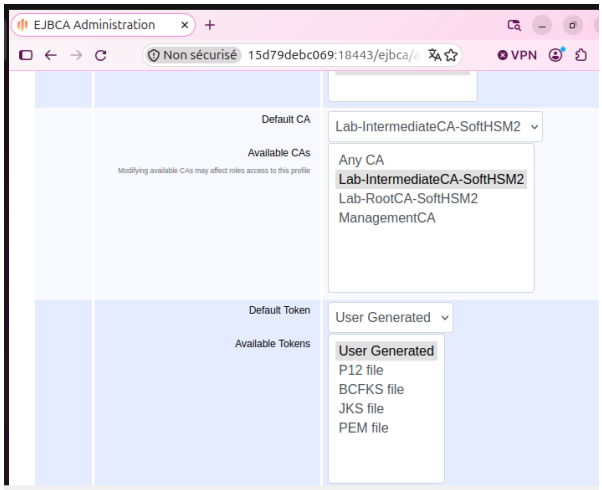
(a) « Default certificate profile must be among available certificate profiles ».



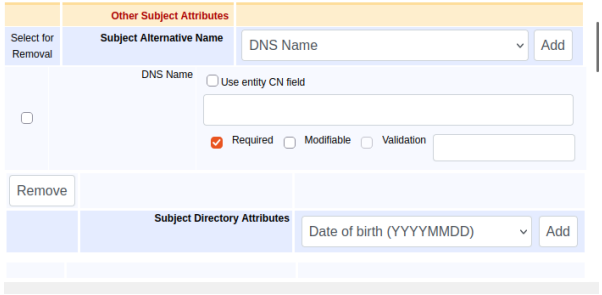
(b) Cause : listes à sélection multiple mal cochées.



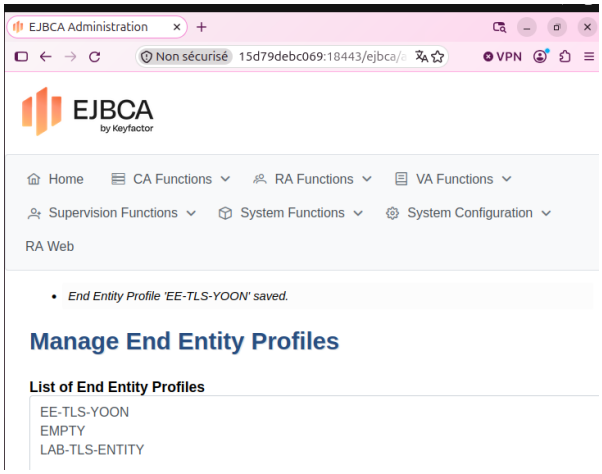
(a) Final : profil et CA uniques.



(b) Final : token *User Generated* uniquement.



(a) DNS Name ajouté (Modifiable à cocher).



(b) Profil enregistré.

Modifiable ou non — O et C sont fixés : personne ne peut demander un certificat au nom d'une autre organisation. CN et DNS Name changent avec l'ALB. *User Generated* garantit que la clé est créée par nous.

12 La CLI AWS dans la VM

Sans les Additions invité, ni copier-coller ni dossier partagé : la CLI AWS a été installée dans la VM. La clé de l'ALB y est créée, utilisée et importée dans ACM sans jamais en sortir.

```
curl -sSLo awscliv2.zip "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip"
unzip -q awscliv2.zip && sudo ./aws/install && rm -rf aws awscliv2.zip
aws --version                # aws-cli/2.39.6
aws login --profile moustapha-login --region eu-west-3 --remote
export AWS_PROFILE=moustapha-login AWS_REGION=eu-west-3 AWS_PAGER=""
aws sts get-caller-identity --query Account --output text      # 231644789895
```

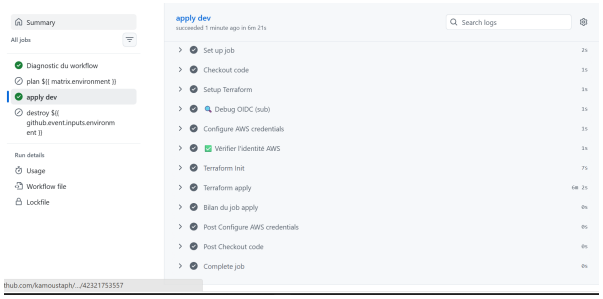
Tentative	Explication
`aws login` sans `--remote`	« authorization has expired » : le navigateur ne s'est pas ouvert à temps
mot de passe saisi dans le terminal	refusé ; le terminal attend le **code de vérification** (`Y29kZT1...`) copié avec le bouton *Copy verification code*
`--remote` + URL ouverte dans Firefox de la VM	**OK**

Mot de passe exposé — Le mot de passe de la console AWS a été écrit en clair dans la conversation : le changer et activer la MFA. Il ne se tape que dans la page AWS, jamais dans un terminal.

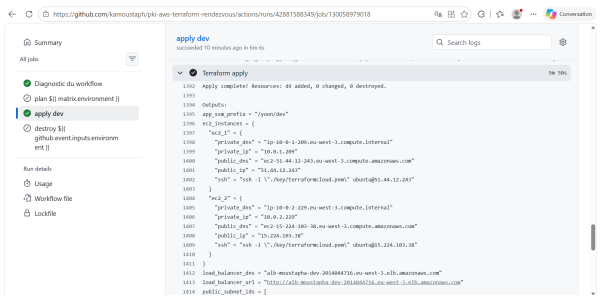
13 Le certificat de l'ALB

13.1 Le nom de l'ALB a changé deux fois

Date	Nom DNS de l'ALB
04/11	`alb-moustapha-dev-755009444...` (détruit)
07/11	`alb-moustapha-dev-1842800432...` (détruit)
10/11	`alb-moustapha-dev-2014044716.eu-west-3.elb.amazonaws.com` (actuel)



(a) Recréation de dev du 07/11 (49 ressources).



(b) Recréation du 10/11 : nom actuel.

```
ALB=$(aws elbv2 describe-load-balancers --names alb-moustapha-dev --query
↪  "LoadBalancers[0].DNSName" --output text)
echo "$ALB"
```

13.2 Nouvelle CRL par la ligne de commande d'EJBCA

La CRL n° 3 expirait le 12/11 : la CLI d'EJBCA a permis de la renouveler sans navigateur.

```
docker exec ejbca-hsm-lab-ejbca-1 /opt/keyfactor/bin/ejbca.sh ca createcrl --caname
↪  "Lab-IntermediateCA-SoftHSM2"
docker exec ejbca-hsm-lab-ejbca-1 /opt/keyfactor/bin/ejbca.sh ca getcrl --caname
↪  "Lab-IntermediateCA-SoftHSM2" -f /tmp/intermediate-ca.crl
docker cp ejbca-hsm-lab-ejbca-1:/tmp/intermediate-ca.crl intermediate-ca.crl
aws s3 cp intermediate-ca.crl s3://pki-moustapha-2026/crl/intermediate-ca.crl
↪  --content-type application/pkix-crl
```

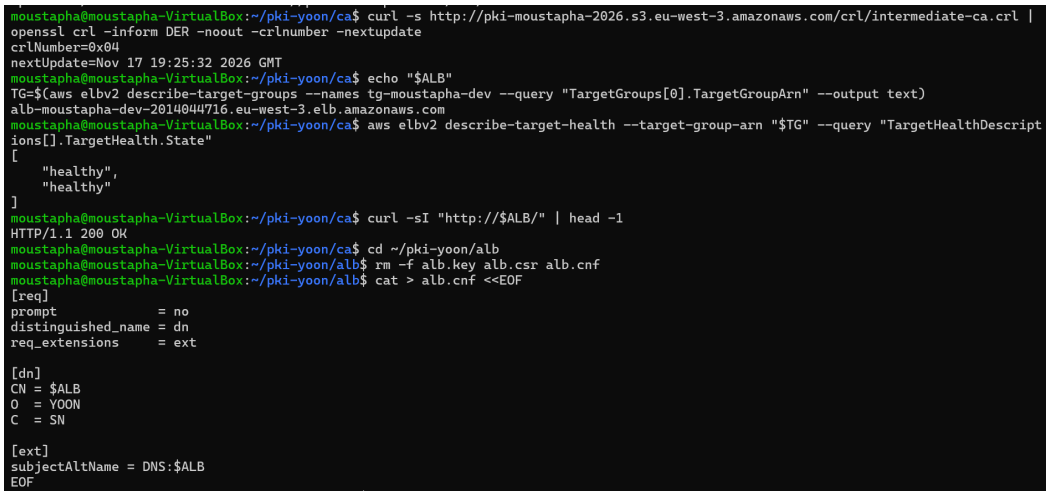


Fig. 13.2 : CRL n° 4 (jusqu'au 17/11) publiée ; cibles `healthy` ; HTTP 200.

13.3 Clé RSA 3072 et CSR

```
cd ~/pki-yoon/alb
cat > alb.cnf <<EOF
[req]
prompt              = no
distinguished_name = dn
req_extensions      = ext
[dn]
CN = $ALB
O  = YOON
C  = SN
[ext]
subjectAltName = DNS:$ALB
EOF
openssl req -new -newkey rsa:3072 -nodes -keyout alb.key -out alb.csr -config alb.cnf
chmod 600 alb.key
```

```
Certificate request self-signature verify OK
subject=CN = alb-moustapha-dev-2014044716.eu-west-3.elb.amazonaws.com, O = YOON, C =
↪  SN
Public-Key: (3072 bit)
DNS:alb-moustapha-dev-2014044716.eu-west-3.elb.amazonaws.com
```

RSA 3072 — La première CSR en 2048 bits a été refusée (`Illegal key length`). Le profil autorisait 3072 : on a choisi 3072, plus robuste. Selon la documentation AWS, ACM importe les clés RSA de 1024 à 4096 bits, et l'Application Load Balancer accepte tous les algorithmes disponibles dans ACM.

13.4 Émission par la ligne de commande d'EJBCA

```
EJBCA="docker exec ejbca-hsm-lab-ejbca-1 /opt/keyfactor/bin/ejbca.sh"
CODE=$(openssl rand -hex 16)          # code d'enrôlement aléatoire, jamais affiché
$EJBCA ra addendentity --username alb-yoon-dev --password "$CODE" \
  --dn "CN=$ALB,O=YOON,C=SN" --altname "dNSName=$ALB" \
```

```
  --caname "Lab-IntermediateCA-SoftHSM2" --type 1 --token USERGENERATED \
  --certprofile TLS-SERVER-YOON --eeprofile EE-TLS-YOON
docker cp alb.csr ejbca-hsm-lab-ejbca-1:/tmp/alb.csr
$EJBCA createcert --username alb-yoon-dev --password "$CODE" -c /tmp/alb.csr -f
↪  /tmp/alb.crt
docker cp ejbca-hsm-lab-ejbca-1:/tmp/alb.crt alb.crt
unset CODE
```

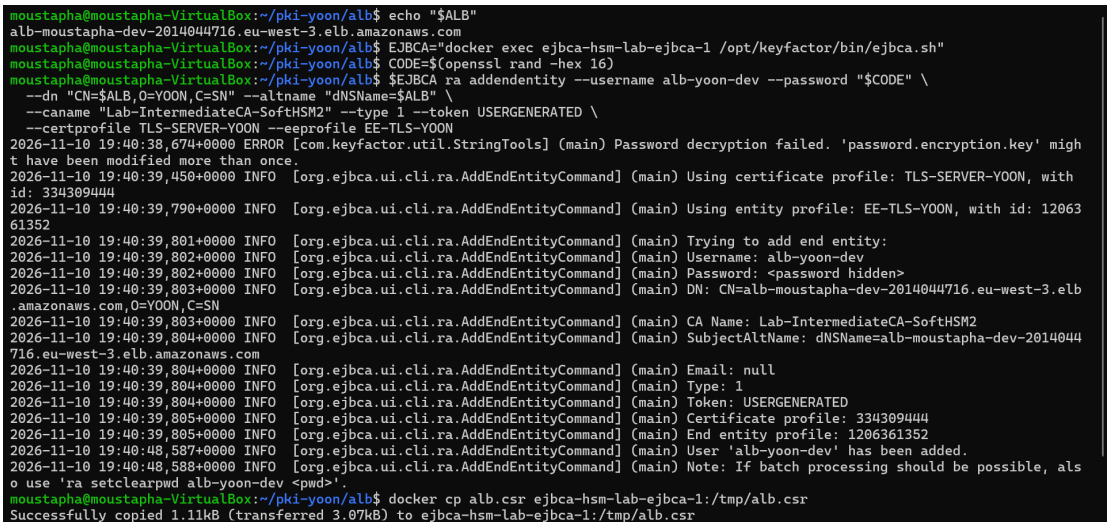


Fig. 13.3 : Création de l'entité `alb-yoon-dev` (le message *Password decryption failed* est sans effet).

Pour réémettre avec la même entité :

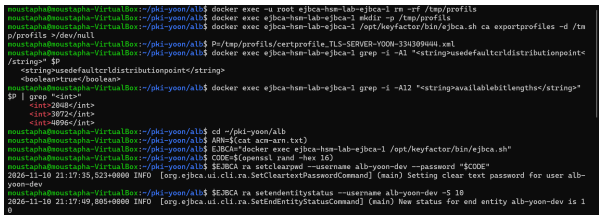
```
$EJBCA ra setclearpwd --username alb-yoon-dev --password "$CODE"
$EJBCA ra setendentitystatus --username alb-yoon-dev -S 10      # 10 = NEW
```

Ligne de commande ou interface ? — Les deux font la même chose. La CLI a été choisie pour voir ce qui est réellement enregistré (l'écran affichait des réglages non sauvegardés), éviter les fautes de frappe sur un nom de 57 caractères, garder la CSR et la clé dans la VM, et pouvoir réémettre en une minute à chaque recréation de dev. L'interface reste l'outil de configuration (CA, profils, services) et de vérification visuelle.

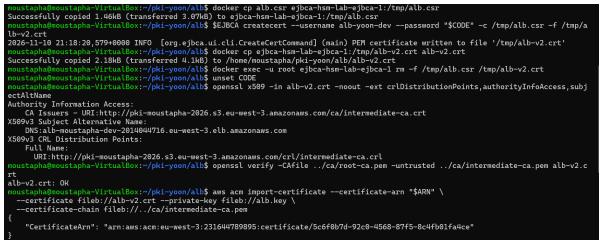
Commande	Équivalent dans l'interface
`ra addendentity`	RA Functions → Add End Entity
`createcert`	RA Web → Enroll → Use Username, puis la CSR
`ra revokecert`	RA Web → Search → Certificates → certificat → Revoke
`ca createcrl` / `ca getcrl`	CA Structure & CRLs → Create CRL / Get CRL
`ca exportprofiles`	aucun : l'interface ne montre pas le contenu enregistré

13.5 Trois émissions successives

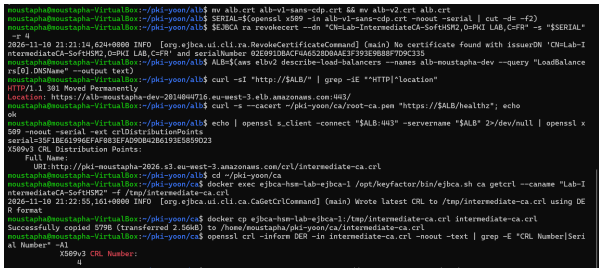
N°	Numéro de série	Problème
1	`02E091DB...C335`	pas d'extension CDP (profil non enregistré)
2	`35F1BE61...5D23`	CDP présent, mais certificat non enregistré dans EJBCA (stockage désactivé)
3	`5D0C16B5...6AAC`	**conforme** : CDP, AIA, enregistré, révocable



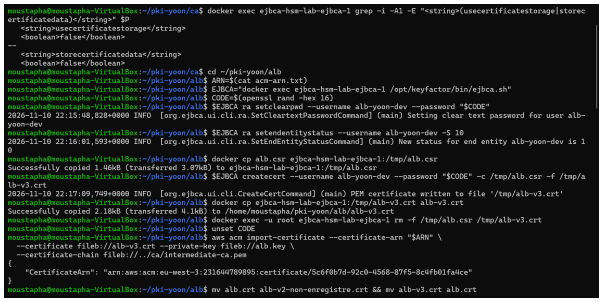
(a) Profil vérifié par l'export, puis réémission.



(b) Certificat n° 2 : CDP présent, réimport sur le même ARN.



(a) Révocation impossible : « No certificate found ».



(b) Certificat n° 3 émis et réimporté.

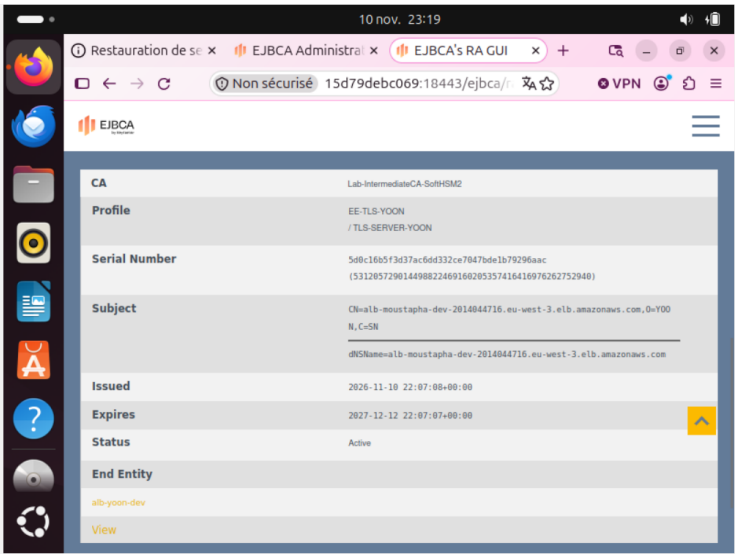


Fig. 13.6 : RA Web : le certificat n° 3 est enregistré et *Active*.

Écran de recherche des entités — Le bouton *Revoke Selected* de *Search End Entities* révoque **tous** les certificats de l'entité, y compris celui en service. Pour un seul certificat, passer par RA Web ou `ra revokecert`.

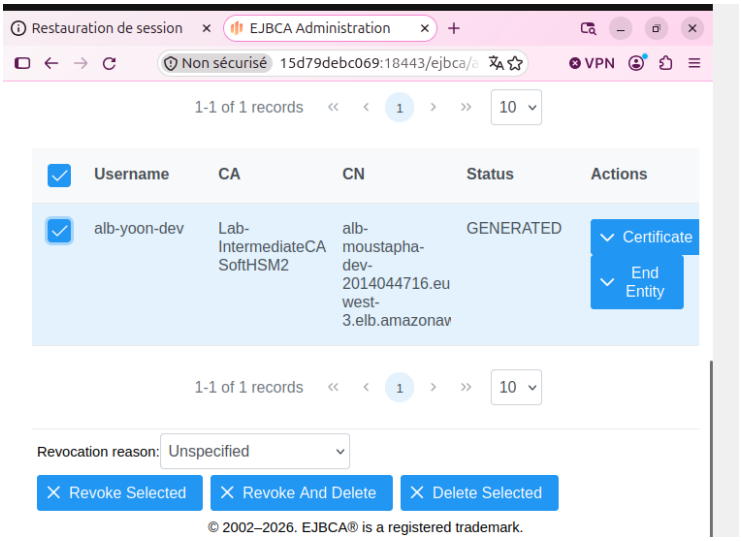


Fig. 13.7 : À ne pas utiliser pour révoquer un seul certificat.

13.6 Vérification et import dans ACM

```
openssl x509 -in alb.crt -noout -ext
↪  subjectAltName,keyUsage,extendedKeyUsage,crlDistributionPoints,authorityInfoAccess
openssl x509 -in alb.crt -noout -pubkey | openssl sha256 ; openssl pkey -in alb.key
↪  -pubout | openssl sha256
openssl verify -crl_check -CRLfile <(openssl crl -inform DER -in
↪  ../ca/intermediate-ca.crl) \
  -CAfile ../ca/root-ca.pem -untrusted ../ca/intermediate-ca.pem alb.crt
ARN=$(aws acm import-certificate --certificate fileb://alb.crt --private-key
↪  fileb://alb.key \
  --certificate-chain fileb://../ca/intermediate-ca.pem \
  --tags Key=Project,Value=moustapha Key=Env,Value=dev Key=Emetteur,Value=EJBCA \
  --query CertificateArn --output text)
echo "$ARN" > acm-arn.txt
# réémissions suivantes : même ARN, aucune modification Terraform
aws acm import-certificate --certificate-arn "$ARN" --certificate fileb://alb.crt \
  --private-key fileb://alb.key --certificate-chain fileb://../ca/intermediate-ca.pem
```

```
"Nom": "alb-moustapha-dev-2014044716.eu-west-3.elb.amazonaws.com", "Type": "IMPORTED",
"Statut": "ISSUED", "Cle": "RSA-3072", "Expire": "2027-12-12T20:47:21+01:00"
ARN : arn:aws:acm:eu-west-3:231644789895:certificate/5c6f0b7d-92c0-4568-87f5-8c4fb01f
↪  a4ce
```

14 Activation du port 443

14.1 Pull request #16

Une seule ligne dans `environments/dev/terraform.tfvars`, fichier suivi par Git et lu par la pipeline :

```
certificate_arn = "arn:aws:acm:eu-west-3:231644789895:certificate/5c6f0b7d-92c0-4568-
↪  87f5-8c4fb01fa4ce"
```

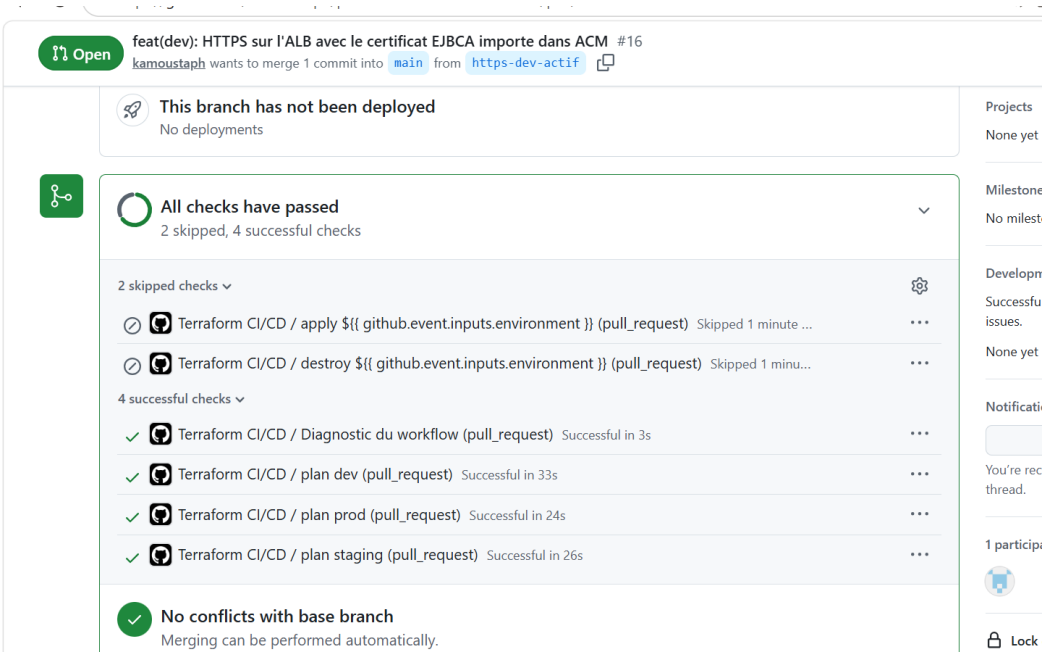
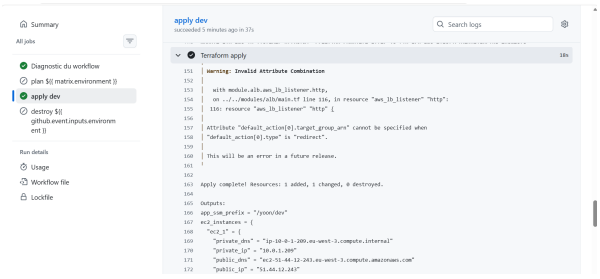
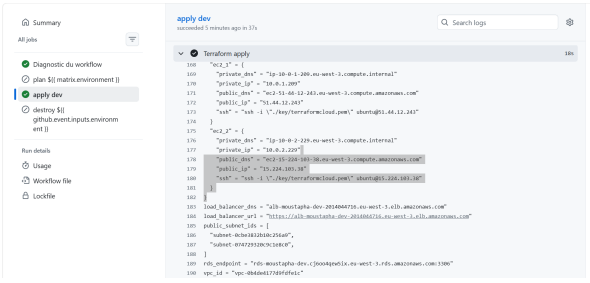


Fig. 14.1 : PR #16 : plans réussis.



(a) `1 added, 1 changed, 0 destroyed`. L'avertissement du provider est sans effet.



(b) `load_balancer_url` passe en `https://`.

14.2 Tests depuis la VM

```
curl -sI http://$ALB/            -> HTTP/1.1 301 Moved Permanently  Location:
↪  https://...:443/
curl --cacert root-ca.pem https://$ALB/healthz   -> ok
openssl s_client ...             -> serial=5D0C16B5F3D37AC6DD332CE7047BDE1B79296AAC +
↪  CDP
```

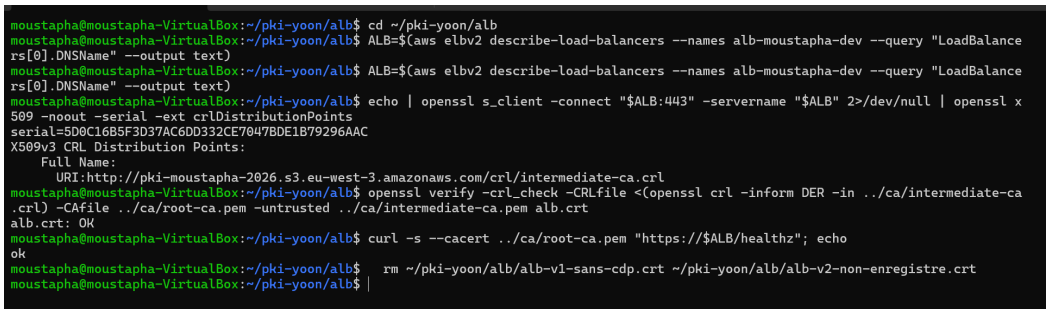


Fig. 14.3 : L'ALB sert le certificat n° 3 ; chaîne et CRL valides.

15 Confiance dans la Root CA et test final

15.1 Windows

```
curl.exe -s -o C:\pki-yoon\ca\root-ca.crt
↪  http://pki-moustapha-2026.s3.eu-west-3.amazonaws.com/ca/root-ca.crt
certutil -user -addstore Root C:\pki-yoon\ca\root-ca.crt     # sans droits
↪  administrateur
certutil -user -store Root "Lab-RootCA-SoftHSM2"
```

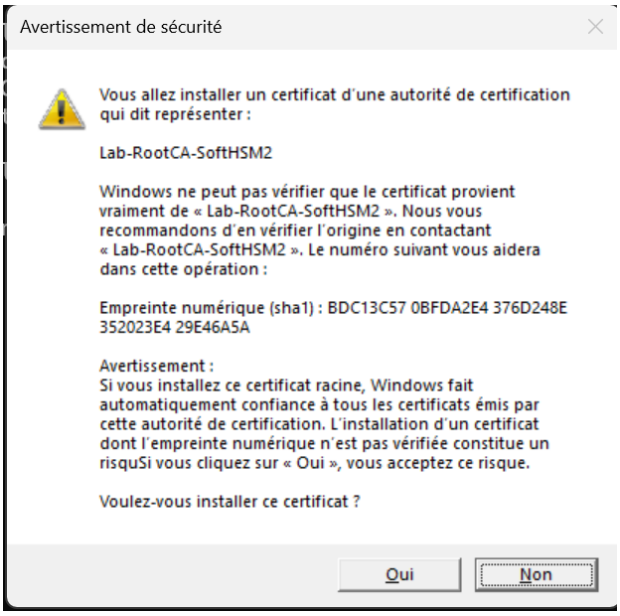


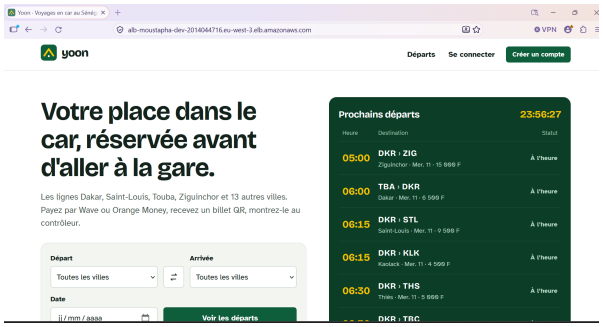
Fig. 15.1 : Avertissement de Windows : vérifier l'empreinte avant d'accepter.

Source	Empreinte SHA-1 de la Root
VM (`openssl x509 -fingerprint -sha1`)	`BD:C1:3C:57:0B:FD:A2:E4:37:6D:24:8E:35:20:23:E4:29:E4:6A:5A`
Windows (`certutil`)	`bdc13c570bfda2e4376d248e352023e429e46a5a`

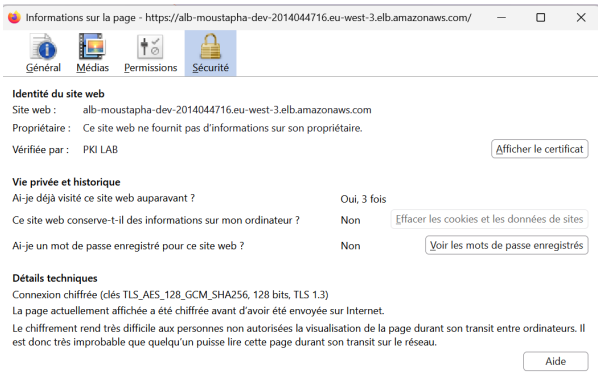
Les empreintes sont identiques : la Root installée est bien la nôtre.

15.2 Vérification dans Firefox

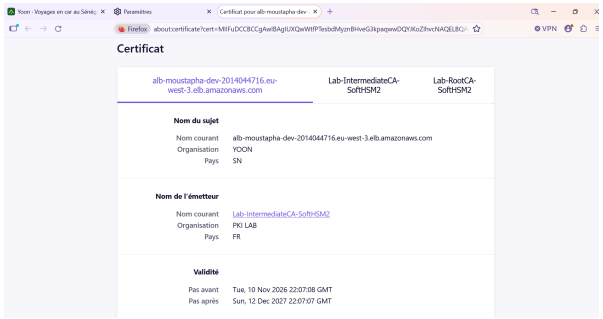
Firefox sous Windows reprend les autorités du magasin Windows.



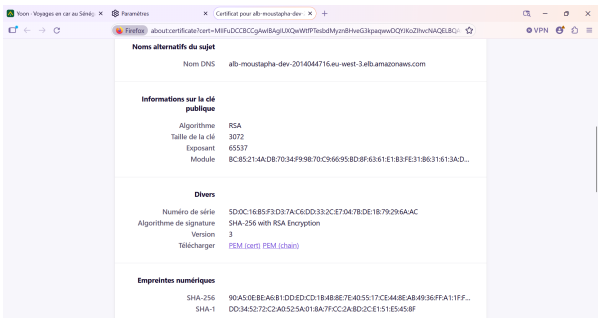
(a) Yoon en HTTPS, sans avertissement.



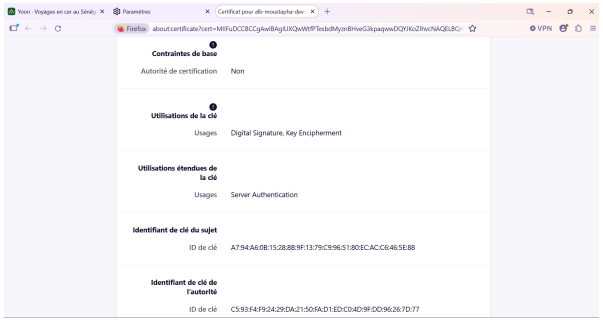
(b) « Vérifiée par : PKI LAB », TLS 1.3.



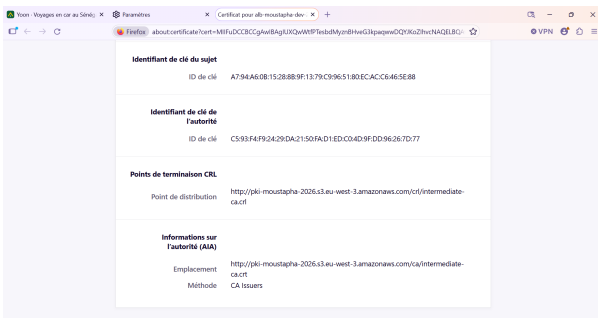
(a) Sujet, émetteur, validité de 397 jours.



(b) SAN, RSA 3072, numéro de série `5D:0C:16:B5:...`



(a) *Autorité de certification : Non* ; usages critiques (marque !).



(b) Points de distribution CRL et AIA vers le bucket.

15.3 Téléphone Android et scanner

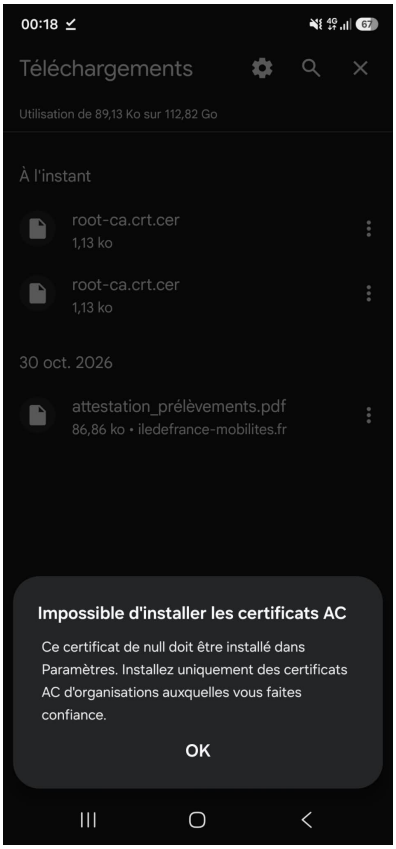


Fig. 15.5 : Android refuse l'installation en touchant le fichier.

Depuis Android 11, une autorité s'installe par les Paramètres : *Sécurité et confidentialité* → *Autres paramètres de sécurité* → *Installer depuis le stockage de l'appareil* → *Certificat AC* → *Installer quand même.*

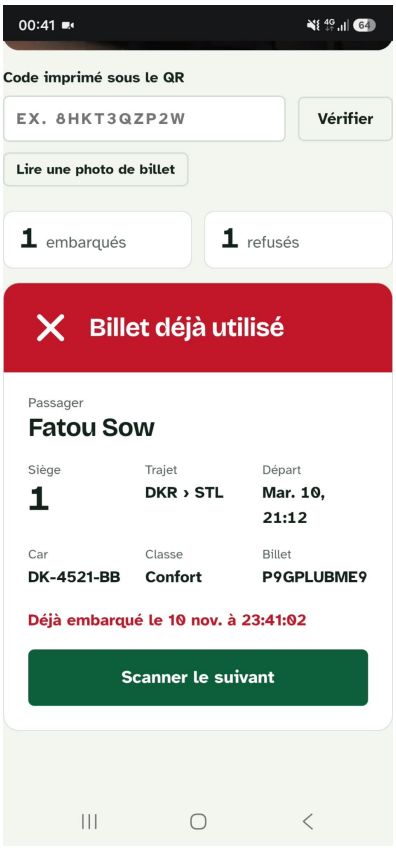


Fig. 15.6 : Test final : le scanner caméra en HTTPS fonctionne. Un billet embarqué, puis refusé au second scan (« Billet déjà utilisé »).

PARTIE III

Sécurité, problèmes connus et exploitation

16 Problèmes connus

16.1 Délai de paiement dépassé dès le départ

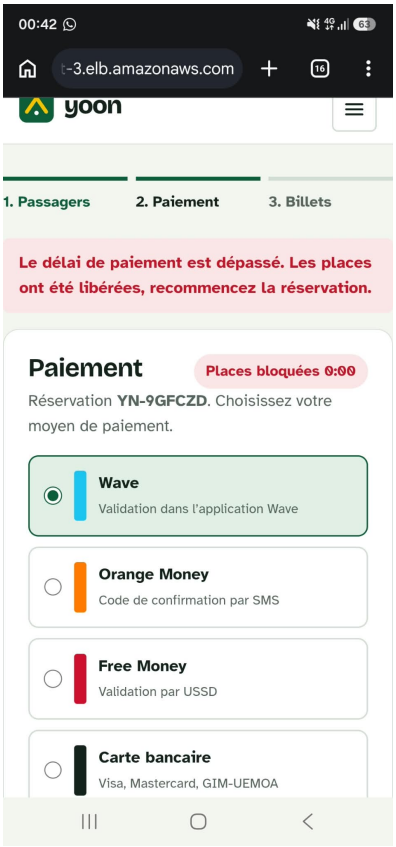


Fig. 16.1 : « Le délai de paiement est dépassé », compteur à 0 :00 dès l'arrivée sur la page.

Diagnostic probable, à confirmer — Le scanner affichait « Déjà embarqué le 10 nov. à 23 :41 :02 » alors que le téléphone indiquait 00 :41 : un décalage d'une heure. Le serveur fonctionne en UTC ; si l'API renvoie l'heure d'expiration sans fuseau, le navigateur en France (UTC+1) la lit une heure trop tôt. Le délai de 15 minutes paraît alors déjà écoulé. Au Sénégal (UTC+0), le problème ne se verrait pas. Correction prévue : renvoyer des dates avec fuseau (`Instant` / `OffsetDateTime`, format ISO 8601 en `Z`).

16.2 Mot de passe de la base dans `terraform.tfvars`

`environments/dev/terraform.tfvars` contient `db_password` en clair, commité dans Git. La pipeline ne l'utilise pas : elle passe `-var="db_password=${{ secrets.DB_PASSWORD }}"`, prioritaire sur `terraform.tfvars`. La valeur en clair reste toutefois publiée dans l'historique.

1. Supprimer la ligne `db_password` (dev, et vérifier staging et prod).
2. Mettre un nouveau mot de passe fort dans le secret GitHub `DB_PASSWORD`.
3. PR, plan (modification en place de RDS et du paramètre SSM), apply.

4. Redéployer les conteneurs : `/opt/yoon/deploy.sh all latest` par SSM, EC2-1 puis EC2-2.

16.3 Autres points

- Avertissement `Invalid Attribute Combination` lors du passage du port 80 en redirection : vérifier qu'il disparaît au prochain plan.
- Mot de passe de la console AWS exposé : à changer, MFA à activer.
- Mots de passe de démonstration publics : à changer avant usage réel.
- Les certificats n° 1 et n° 2 n'ont pas pu être révoqués (non enregistrés) ; leurs fichiers locaux ont été supprimés. Leur clé est celle du n° 3, protégée dans la VM et dans ACM.
- Le service EJBCA *CRL Updater* reste à confirmer.

17 Exploitation

17.1 Publier la CRL chaque semaine

La CRL de l'Intermediate vaut 7 jours. Avant chaque *nextUpdate* :

```
export AWS_PROFILE=moustapha-login AWS_REGION=eu-west-3 AWS_PAGER=""
cd ~/pki-yoon/ca
docker exec ejbca-hsm-lab-ejbca-1 /opt/keyfactor/bin/ejbca.sh ca createcrl --caname
↪  "Lab-IntermediateCA-SoftHSM2"
docker exec ejbca-hsm-lab-ejbca-1 /opt/keyfactor/bin/ejbca.sh ca getcrl --caname
↪  "Lab-IntermediateCA-SoftHSM2" -f /tmp/intermediate-ca.crl
docker cp ejbca-hsm-lab-ejbca-1:/tmp/intermediate-ca.crl intermediate-ca.crl
openssl crl -inform DER -in intermediate-ca.crl -noout -crlnumber -nextupdate
aws s3 cp intermediate-ca.crl s3://pki-moustapha-2026/crl/intermediate-ca.crl
↪  --content-type application/pkix-crl
```

La CRL de la Root (30 jours) se publie de la même façon, avec `Lab-RootCA-SoftHSM2` et `crl/root-ca.crl`.

17.2 Si dev est recréé

Le nom de l'ALB change : nouvelle clé et CSR, émission (`setclearpwd`, `setendentitystatus -S 10`, `ra addendentity` si besoin avec le nouveau DN, `createcert`), **import sans** `--certificate-arn` (nouvel ARN), mise à jour de `certificate_arn` dans `terraform.tfvars`, PR, apply. Un script automatisant ces étapes est prévu.

17.3 Renouvellement

30 jours avant le 12/12/2027 : réémission avec la même entité, puis réimport **sur le même ARN** (aucun changement Terraform).

17.4 Révoquer un certificat

```
SERIAL=$(openssl x509 -in cert.crt -noout -serial | cut -d= -f2 | sed 's/^0*//')
$EJBCA ra revokecert --dn "CN=Lab-IntermediateCA-SoftHSM2,O=PKI LAB,C=FR" -s "$SERIAL"
↪  -r 4
```

Puis publier la nouvelle CRL. EJBCA enregistre le numéro de série sans zéro initial.

18 Règles de sécurité

Élément	Où il vit	Jamais
Clés des Root et Intermediate	SoftHSM, dans la VM	exportées, copiées, dans Git
`alb.key`	ACM (chiffrée) et `~/pki-yoon/alb` (600)	dans Git, sur S3, dans Terraform
`SuperAdmin.p12`, PIN SoftHSM	`~/pki-lab-backups/admin`, coffre de mots de passe	dans un fichier du projet
`key/terraformcloud.pem`	ignoré par `*.pem`	commité (vérifié : jamais dans l'historique)
Secrets de l'application	SSM SecureString	dans une image ou un `.env` commité
Certificats de CA, CRL	bucket public	— (publics par nature)

A Récapitulatif

Élément	Valeur
ALB	`alb-moustapha-dev-2014044716.eu-west-3.elb.amazonaws.com`
Certificat ACM	`...certificate/5c6f0b7d-92c0-4568-87f5-8c4fb01fa4ce`, RSA 3072, jusqu'au 12/12/2027
Certificat en service	n° `5D0C16B5F3D37AC6DD332CE7047BDE1B79296AAC`
Entité EJBCA	`alb-yoon-dev`, profils `EE-TLS-YOON` / `TLS-SERVER-YOON`
Root CA	`Lab-RootCA-SoftHSM2`, SHA-1 `BD:C1:3C:57:...:6A:5A`, jusqu'au 13/10/2036
Intermediate CA	`Lab-IntermediateCA-SoftHSM2`, jusqu'au 15/10/2031, CRL 7 jours
Bucket PKI	`pki-moustapha-2026`
Profil CLI dans la VM	`moustapha-login` (`terraform-user-moustapha`)
Conteneurs EJBCA	`ejbca-hsm-lab-ejbca-1`, `ejbca-hsm-lab-mariadb-1`
Instantané de sécurité	`pki-avant-fsck-2026-11-06`

Pull request	Contenu
Bakend #1, Frontend #1	refonte de l'application
Bakend #2, Frontend #2	publication des images dans ECR
Terraform #13, #14	serveurs Docker, module `app`, correctif d'ordre de démarrage
Terraform #15	bucket des CRL, HTTPS optionnel sur l'ALB, documentation
Terraform #16	activation du HTTPS en dev (`certificate_arn`)

B Dépannage

Symptôme	Cause et correction
`AssumeRoleWithWebIdentity` refusé	`sub` au format avec identifiants : l'afficher et corriger la trust policy
ALB en 502	conteneurs non lancés : `deploy.sh` par SSM
CLI AWS bloquée sur `:`	`$env:AWS_PAGER = ""`
`Copy-Item` : chemin introuvable	créer le dossier avec `New-Item -ItemType Directory -Force`
`BlockPublicPolicy`	blocage public au niveau du compte
`LoadBalancerNotFound`	dev détruit : recréer, puis réémettre le certificat
`Illegal key length ... 2048`	bits non autorisés dans le profil, ou clé 3072
Certificat sans CDP	*Use CA defined CRL Distribution Point* non enregistré
`No certificate found` à la révocation	stockage désactivé, ou zéro initial du numéro de série
`rm: Operation not permitted` dans le conteneur	`docker exec -u root`
Android : « Impossible d'installer les certificats AC »	installer par les Paramètres
Écran noir de la VM	disque incohérent (`fsck`) ou module `vboxguest` (GRUB `module_blacklist`)
`aws login` : code invalide	coller le code de vérification, pas le mot de passe